CODE READING GUIDE / 2026.11.08

Kokomi Kernel

構造と動作の詳解

Live会話から、身体・世界モデル・記憶まで

何が起きると、どの部品が動くのか。

なぜ、その機能が必要なのか。

現在の実装を、コードの根拠とともに読み解く。

対象: codex/live-responses-kernel の作業ツリー

HEAD: cbb47cf32ec3d778abe49c592092c738996cf4fe

未コミット変更を含む、現在の構造を読む資料

日本語 / 図解・動作例・用語集・実装参照付き

この資料の使い方

全体の構造を自分の言葉で説明できるようになることを目的に、現在のコードを入力から出力まで追った資料です。最初から全ファイルを暗記する必要はありません。まず「会話」「身体状態」「操作」「記憶」の流れを分け、必要なときに参照先へ戻ってください。

おすすめの読み順

1周目は01〜03、07〜11、27〜29、34、39〜43を読みます。全体像と、会話・身体・記憶の責任がつながります。2周目は15〜26と38〜49で、情報の形式と保存を追います。3周目は動作例51〜54をコードと照らし、用語集60〜62と実装参照表67〜68を辞書として使います。

記述の意味

「実装」はコードで確認した動作、「設定」は同梱config.jsonの値、「例」は理解用に組み立てた架空の入力です。「理由」は責任分割や障害対策から読み取れる設計上の意図を説明します。実機で確かめた結果と、コードから分かる結果を混同しません。

対象と調査方法

2026年11月8日（日本時間）の作業フォルダーを調査しました。未コミット変更を含むソース・設定・共有制御プロンプト・既存テストが対象です。同梱Node.js 24.22.1で84テストの成功を確認しました。実APIの有料会話や実機を動かす検証は行っていません。個人の会話・記憶の内容や.envの秘密値も転載していません。

参照の使い方

各ページ末尾の「根拠」はリポジトリ内のファイル名と行番号です。未コミット変更の内容も固定できるよう、参照対象のSHA-256一覧を付属ファイルへ保存しています。目次のリンクとしおりから各話題へ移動できます。前提知識なしで読み始められるよう、現在の構成だけを説明します。

> 一番大切な区別は、観測・解釈・提案・実行結果です。この四つを分けると、複雑に見える実装をかなり読みやすくできます。

目次

01 現在地：この資料が説明するカーネル	6
02 全体像：会話・身体・記憶がつながる道	7
03 誰が何を決めるか：責任と権限	8
04 ファイル地図：部品の配置をつかむ	9
05 起動：保存された事実を復元する	10
06 共有制御プロンプト：会話と判断に同じ能力を伝える	11
07 普通の会話：音声と文字起こしは同時に進む	12
08 会話開始：接続の準備と開始確認	13
09 新しい会話の境界：過去は参考資料として渡す	14
10 身体情報の同期：定性的な状態を会話へ渡す	15
11 環境の数値を内側に保つ：情報の投影	16
12 文脈の予算：全部を毎回入力しない	17
13 会話終了：手元の音声停止と相手側の確認	18
14 音声の警告と診断：何が途切れたのかを調べる	19
15 観測：外からの報告を共通形式へそろえる	20
16 状態：観測を適用して「いま」を作る	21
17 鮮度：unknown・stale・falseの意味	22
18 世界モデル：数値から扱いやすい意味へ	23
19 センサーの経路：毎秒の更新と保存の間隔	24
20 人物の情報：存在・追跡・名前を区別する	25
21 接触と音分類：身体の周囲を意味へ変える	26
22 ドライブ：行動選択のための必要度	27
23 自発行動：候補の生成と依頼送信を分ける	28
24 認知文脈：判断へ渡す一回分の資料	29

目次（続き）

25 Responsesの呼出し：一回の判断をどう依頼するか 30
26 形式と値の監査：スキーマとカーネルの役割 31
27 委譲の処理：依頼を確定して結果を返す 32
28 独立した待ち行列：制御と記憶を並行させる 33
29 訂正と取り消し：古い判断を実行しない 34
30 カメラ質問：二つの入口を一つの確認へまとめる 35
31 画像の渡し方：撮影から説明まで 36
32 Web検索：公開情報を確認する専用の仕事 37
33 行動ゲート：操作案の実行条件を確認する 38
34 フィードバック：意図と結果を別に記録する 39
35 LEDと涙：成功が示す物理的な範囲 40
36 公開投稿と人物登録：明示的な許可を必要とする操作 41
37 情報照会と文字入力：入口の完了条件 42
38 会話ログ：原文と処理位置を保存する 43
39 記憶の種類：ログ・要約・採用済みの事実 44
40 記憶バッチ：どの会話を一度に処理するか 45
41 根拠の受渡し：短い参照から原文を復元する 46
42 抽出と照合：記憶の本文を二段階で確かめる 47
43 自動採用：確信度だけではacceptedにしない 48
44 検索と統合：関連する記憶を少数だけ使う 49
45 チェックポイント：途中の保存から再開する 50
46 連続失敗：再試行の間隔と自動処理の保留 51
47 社会状態：関係・境界・約束を保持する 52
48 保存と復元：何を残し、何を計算し直すか 53

目次（続き）

49 API診断：通信成功とカーネル採用を分ける 54
50 ダッシュボード：運用者が経路を追う入口 55
51 動作例：LEDを青にして、その途中で訂正する 56
52 動作例：記憶の抽出中に、目の前の物を質問する 57
53 動作例：公開情報の検索と、部屋の暑さの確認 58
54 動作例：「覚えて」と言ってから保存が確定するまで 59
55 HTTP一覧：入口ごとの役割と完了条件 60
56 設定一覧：動作を決める主な数字 61
57 検証：テストで確認できたことと、その範囲 62
58 現在の実装の範囲：万能な知能や完全な復旧ではない 63
59 コードの読み方：入力から結果まで一本の線を引く 64
60 用語集I：観測・状態・世界モデル・認知・身体 65
61 用語集II：行動・フィードバック・根拠・記憶 66
62 用語集III：接続・API・並行処理・復旧 67
63 データ例：観測・世界・判断文脈の形 68
64 データ例：制御の提案と、実行結果の形 69
65 データ例：短い根拠参照と本文の照合 70
66 IDの地図：どの出来事と仕事を結び付けるか 71
67 実装参照I：状態・世界・行動・保存 72
68 実装参照II：会話・Responses・記憶処理・画面 73
69 なぜこの機能が必要か：起きる問題から読む 74
70 出典・再現性：この説明が指す作業ツリー 75

01 現在地：この資料が説明するカーネル

このプロジェクトのカーネルは、身体を持つ対話システムで、センサー、会話AI、長期記憶、機体操作をつなぐNode.jsアプリケーションです。起点はalice.jsです。外から何が報告されたかを整理し、現在の状態を保持し、実行してよい操作かを確認して、結果を次の判断へ戻します。

対象は2026年11月8日の**codex/live-responses-kernelの作業ツリー**です。HEADは cbb47cf32ec3d778abe49c592092c738996cf4feです。未コミットのファイル内容も対象に含みます。この一冊では、現在の構成と動作を最初から説明します。

領域	現在の実装・設定	何を担当するか
音声会話	GPT-Live、gpt-live-1	聞く、話す、処理の委譲を選ぶ
判断・画像・検索	Responses API、gpt-6-luna	文脈から操作案・確認結果を返す
状態と世界モデル	カーネル内の規則	観測、鮮度、温熱・明暗・在室を管理
行動制御	形式監査とActionGate	権限、回数、重複、間隔、結果を管理
記憶処理	会話ログ、抽出、照合、採否	根拠のあるユーザーの事実を保存
自発行動	候補生成あり、自動送信は無効	任意に検討できる振る舞いを作る
室内環境の会話	定性的な世界の状態	hot、darkなどをモデルへ渡す

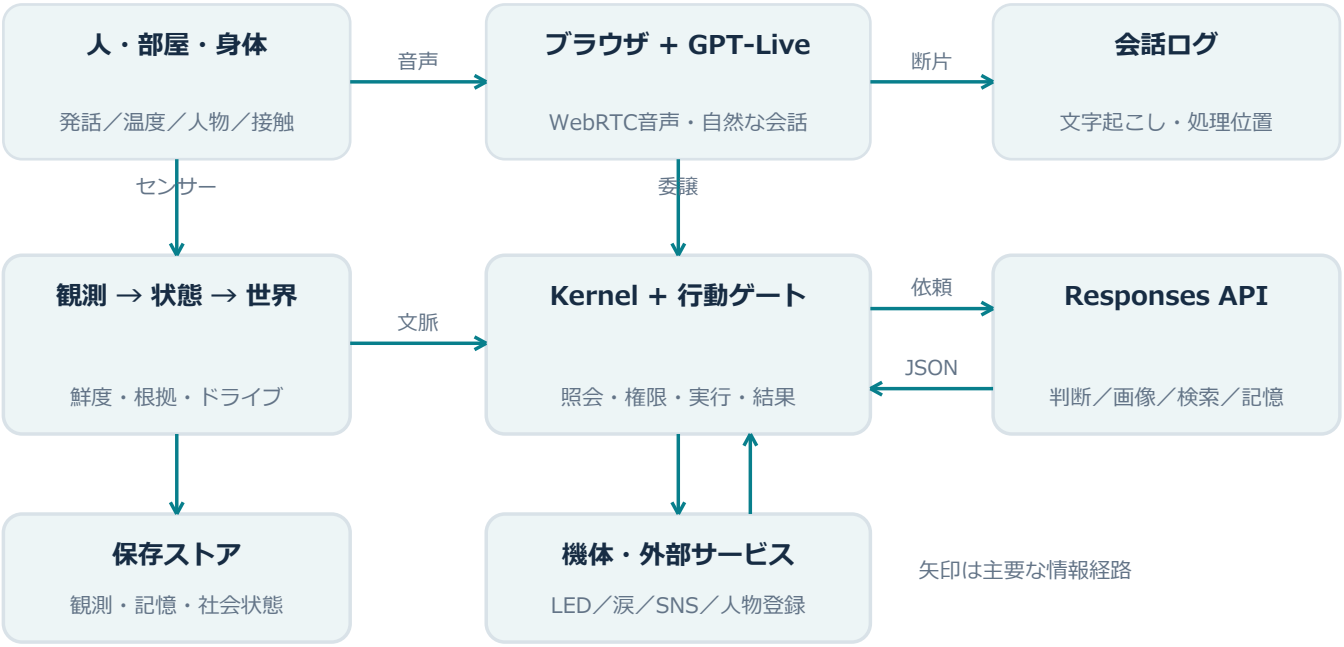
確認した範囲

ソース、設定、共有制御プロンプト、既存テストを照合しました。同梱Node.js 24.22.1で84テストが成功しています。試験ではAPIや機体を仮の接続先へ置き換えています。実アカウントの有料音声会話や物理機構の動作完了を、この調査で測定したわけではありません。

> 「その機能のコードがある」「現在の設定で動く」「実環境で成功した」は別々です。この資料では、コードから確認できる条件と成功の意味を具体的に説明します。

根拠（作業ツリー）：alice.js / config.json / package.json / test/live-runtime.test.js

02 全体像：会話・身体・記憶がつながる道



音声の主要経路は、**ブラウザのマイク → GPT-Live → ブラウザのスピーカー**です。カーネルはサーバー側のsideband接続から会話の文字起こしや委譲イベントを受けます。会話を続けることと、身体側で仕事を進めることが並行します。

身体側の道は、**観測 → 状態 → 世界モデル → 判断用文脈 → Responses → 監査 → 操作 → 結果**です。モデルが操作案を生成しても、それだけで機体は動きません。カーネルが形式と実行条件を確認し、実行先へ命令を送り、返ってきた結果を記録します。

記憶は、会話ログから別の処理で作ります。Responsesが候補を抽出し、カーネルが原文の根拠を復元して、別の依頼で本文の支持を照合します。採用したものだけを、関連する話題で再検索します。

なぜこの分割が必要か

自然な会話には短い応答時間が必要で、機体操作には根拠・権限・結果確認が必要です。二つを同じ速度で処理する必要はありません。画像の確認や記憶抽出を待つ間も会話は進め、操作の成否についてはカーネルが確認できた範囲を返す構成です。

根拠（作業ツリー）：alice.js:362 / src/live/kernel-bridge.js:137 / src/live/memory-extractor.js:109

03 誰が何を決めるか：責任と権限

全体を一つの「AI」と考えると、どの情報を信頼してよいかが分かりにくくなります。担当を分けて見ると、文章の解釈、事実の管理、機体の実行が別の役割であることが見えます。

担当	持つ情報・権限	具体例
外部センサー	生の観測を報告	温度32℃、人物track 7が見えた
StateStore	最新値、時刻、根拠、鮮度	温度の報告がまだ使えるか
WorldModel	規則で導く意味	現在の設定ではhot
DriveSystem	行動選択の必要度	温熱快適性を考える度合い0.9
GPT-Live	会話と委譲の選択	カメラ確認が必要なので依頼する
ResponsesBackend	モデル呼出しと出力の受取	JSON、画像説明、検索回答
LiveKernelBridge	依頼の組立てと処理の進行	照会結果を次の判断へ渡す
形式監査・ActionGate	操作案の許否	形式・今回の許可・重複を確認
実行アダプター	外側のサービスを呼ぶ	LEDや人物登録へHTTPを送る
保存ストア	採用済み記録と履歴	記憶、境界、意図と結果

「提案」と「事実」の違い

行動候補は、検討してもよい選択肢です。操作案は、モデルが実行を求めた内容です。acceptedの記憶は、保存の条件を通った情報です。outcomeは、実行後にカーネルが得た結果です。どれも同じ重みの情報ではありません。

操作を許可する境界と、声に出す内容を扱う境界も同じではありません。Liveの音声は自然に生成され、すべての音声をActionGateへ通してから再生する経路ではありません。特に社会状態の禁止がどこに効くかは、47で説明します。

根拠（作業ツリー）：src/behavior/index.js:26 / src/live/responses-backend.js:106 / src/behavior/action-gate.js:57 / src/social/social-state-store.js:136

04 ファイル地図：部品の配置をつかむ

alice.jsは、サービスを作り、HTTPの入口を用意し、各部品の呼出しをつなぎます。src以下は主な責務ごとに分かれています。個々のクラスだけでなく、それをalice.jsがどの引数で組み合わせるかを読むことが大切です。

場所	責務	主な入口
alice.js	起動、HTTP、ポーリング、実行先	ingestObservation、executeGatedAction
src/behavior/	観測、状態、世界、必要度、候補	BehaviorArchitecture.observe
src/live/session-service.js	Live接続と文脈同期	start、handle、sync、stop
src/live/kernel-bridge.js	委譲、画像、照会、記憶の進行	delegate、process、routeCamera
src/live/responses-*.js	API呼出し、JSONスキーマ	ResponsesBackend、responseFormats
src/live/memory-*.js	抽出、根拠の復元、再試行	MemoryExtractor、memoryConversation
src/live/conversation-store.js	会話の原文と処理位置	append、batch、prepare、commit
src/protocol/	返ってきた出力の監査	LlmResponseProtocol.audit
src/memory/ / src/social/	長期記憶と社会状態	MemoryStore、SocialStateStore
dashboard/	管理画面と音声クライアント	LiveBrowserClient、LiveAudioMonitor
control_prompt.md	会話と判断の共有方針	LiveとResponsesの役割を分ける
config.json / test/	実行条件と検証契約	閾値、間隔、既存テスト

リポジトリの外側

カメラのsnapshot、顔の照合・登録、LED・涙の機構、環境センサーのHTTPサービスは外部です。カーネルが持つのは、それらに接続する契約と結果の扱いです。ドライバーや機構の内部まで、このNode.jsが実装しているとは限りません。

根拠（作業ツリー）：alice.js / src/behavior/index.js / src/live/kernel-bridge.js / config.json

05 起動：保存された事実を復元する

起動時にconfig.jsonと環境変数を読み、状態、記憶、社会状態、API接続用の部品を作ります。control_prompt.mdは起動時に一度全文を読み込みます。Liveの音声セッションは、この段階で自動作成されません。



runtimeReadyの役割

記憶、社会状態、観測ログの読込を待ち、観測を順番に再適用します。generateProposals=falseで再生するので、保存された過去の人物到着を、新しい挨拶候補として作り直しません。未完了の操作意図が残れば、結果が確認できなかった状態として閉じます。

会話ログと記憶抽出のcheckpointはconversationReadyで別に読みます。両方の準備が終わり、未処理の会話があれば抽出を予約します。保存されているpreparedがあれば、その結果を再利用できます。

起動できることとAPIが使えること

キーがなくてもHTTPとダッシュボードは起動でき、欠けている設定を確認できます。Live開始とResponses呼出しにはOPENAI_API_KEYが必要です。サービスのready表示は主にキーの存在に基づくため、実アカウントの権限やネットワークの成功まで先に証明した表示ではありません。

根拠（作業ツリー）：alice.js:67 / alice.js:110 / alice.js:125 / src/behavior/action-gate.js:125

06 共有制御プロンプト：会話と判断に同じ能力を伝える

control_prompt.mdは、システムの能力と制約の原典です。LiveにもResponsesにも全文を渡しますが、先頭のExecution roleで役割を指定します。人格ファイルの読込や人格IDの付加は行わず、自然でニュートラルな日本語会話を指定しています。

受け手	指定する役割	その役割での動き
GPT-Live	Live voice frontend	会話し、必要な処理を委譲する
通常制御・画像のResponses	Responses backend	taskに合ったJSONを返す
記憶抽出・照合	専用の短いinstructions	記憶の抽出・支持確認だけを行う
Web検索	検索専用instructions	公開情報を出典付きで確認する

Liveには、カメラ、LED、涙、投稿、人物登録、状態照会、記憶、検索の能力を知らせます。声側が直接画像を受け取らなくても、システムとして画像確認ができることを理解させるためです。JSONの出力規則はバックエンド向けであり、声でturn_idなどを読み上げる指示ではありません。

いつ反映されるか

Live用の全文はセッション作成時のinstructionsへ入ります。会話中に同じ全文を繰り返し送る処理ではありません。Responsesの制御・画像解析には、各リクエストのinstructionsとして送ります。記憶と検索は目的に応じた専用指示を使います。

編集を反映するにはカーネルを再起動し、新しい音声会話を開始します。Live開始前には、共有全文・役割指定・身体由来の初期話し方の合計が16,384バイト以内かを確認します。超過時は能力説明を途中で切らず、作成を失敗させます。

> プロンプトはモデルに与える方針です。操作回数や値域を強制するのは、後段のスキーマ・監査・ゲートです。両方の役割を確認すると、どこまでがコードの保証か分かります。

根拠（作業ツリー）：alice.js:110 / alice.js:369 / src/live/session-service.js:60 / control_prompt.md / test/live-runtime.test.js

07 普通の会話：音声と文字起こしは同時に進む

「今日は疲れた」と話しかけた場合、ブラウザの音声トラックからLiveへ入力し、生成された音声をAudio要素で再生します。その会話だけで応じられる場合は、共有プロンプトで委譲しない方針です。これはモデルの判断への指示であり、すべての発言をコードが完全分類しているわけではありません。

サーバーで同時に起きること

sidebandは、ユーザーと助手の文字起こしの断片を受け取ります。ConversationStoreへ、発言者、セッション、開始・終了位置、受信時刻を付けて保存します。画面もデータチャネルから文字起こしを受け、利用者に表示します。

userの断片が保存されると、interaction.live_activityを取り込み、最近のやり取りの時刻を更新します。これにより、会話しているのにsocial_contactが無交流として増え続けることを避けます。断片ごとに通常の操作判断を必ずAPIへ依頼する処理ではありません。

もう一つ、安定したユーザー発話がカメラ質問かを確かめる入口も呼びます。対象の短い質問に合致したときだけ、読み取り専用の画像経路へ進みます。詳しくは30を参照してください。

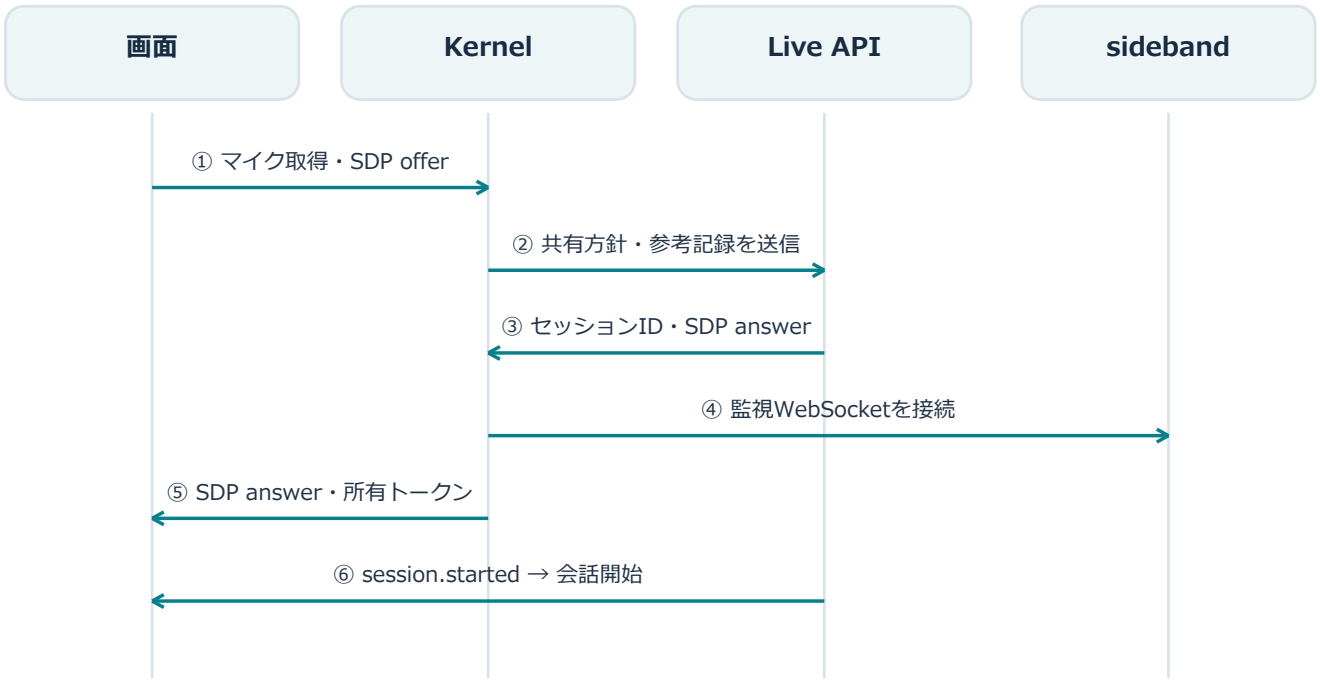
その場で覚えていることと長期保存

Liveが現在の会話を参照できること、原文がログにあること、acceptedの長期記憶になったことは別です。長期記憶には抽出、根拠復元、照合、採否の段階があります。会話に一度出ただけで、次回も必ず検索される記憶として確定するわけではありません。

ブラウザはechoCancellation、noiseSuppression、autoGainControlを要求します。実際のマイクとスピーカーの配置やOSでの効果は、別途音響の確認が必要です。文字起こしがあることだけで、声がすべて正しく再生されたとは判断しません。

根拠（作業ツリー）：dashboard/live-client.js:23 / src/live/session-service.js:208 / alice.js:387 / src/behavior/drive-system.js:19

08 会話開始：接続の準備と開始確認



開始ボタンでマイクを取得し、PeerConnectionへトラックとoai-eventsデータチャネルを加えます。SDPは接続条件の記述、ICEは接続に使える経路を集める手続きです。ICE待ちは既定10秒です。サーバーへSDP offerを渡し、Liveの接続条件を交換します。

サーバーは共有制御プロンプト、初期の話し方、保存済みの参考情報を準備し、/v1/live/sessionsへPOSTします。モデルはgpt-live-1、声はharema、委譲はclient、store=falseです。作成要求の期限は15秒です。

同じセッションへ二つの接続

セッションIDとSDP answerを受けると、カーネルはattach用のsideband WebSocketを作ります。ブラウザへはSDP answerと、32バイトの乱数から作る所有トークンを返します。APIキーはサーバーに保持します。

sidebandのopenだけではactiveにしません。session.started、またはそれを受けた所有ブラウザのheartbeatを待ちます。開始前に文脈appendを送らないことで、音声準備と制御の順序をそろえます。

開始途中で終了が押された場合は、generationで古い処理を無効化します。遅れて作られたセッションにもstopを送り、二重の会話が残りにくくします。サーバーも一つの未終了セッションがあれば、新規作成を409で拒否します。

根拠（作業ツリー）：dashboard/live-client.js:23 / src/live/session-service.js:49 / src/live/session-service.js:224 / src/live/http-routes.js

09 新しい会話の境界：過去は参考資料として渡す

セッションを作るときには、前回の要約、現在の身体状態、関連する採用済み記憶、最近の会話を使えます。ただし、昨日の検索依頼や返答を「今も進行中の仕事」として再開しないよう、新しい会話の始まりを明示します。

初期input	入れるもの	読ませる意味
userの参考資料	要約、身体状態、採用済み記憶	今回の新しい依頼ではない
同資料内の過去会話	recorded_at、speaker、text	時刻と話者の付いた過去の記録
developerの開始指示	今回のUTC開始時刻、挨拶、待機	短く挨拶し、新しい発話を待つ

過去会話を、今回のuser/assistantのやり取りとしてそのまま再演するのではなく、JSONの参考行としてまとめます。直近最大24グループを候補とし、予算に入る新しい記録を優先します。kernelの内部結果は、会話履歴の行としては加えません。

開始指示の内容

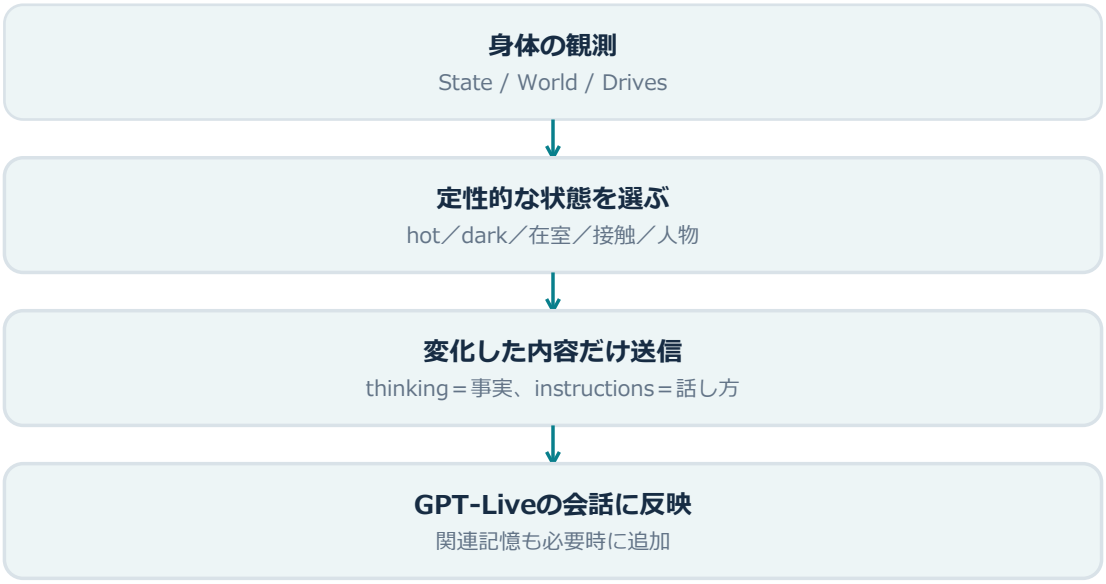
「以前の依頼や検索は終了している」「過去の返答の続きを話さない」「ユーザーが改めて話題にしたとき参考にする」と伝えます。開始境界の文を入れる予算を先に確保し、足りなければ作成前に失敗します。

数値付きの環境描写を検出した履歴・要約・記憶は、初期資料への追加から外します。測定時刻が古い温度を、新しい会話で現在値として使わせないためです。原文の会話ログ自体をこの処理で書き換えるわけではありません。

> 参考資料を引き継ぐことと、未完了の操作権限を引き継ぐことは別です。今回の実行は、新しい依頼と現在の条件で判断します。

根拠（作業ツリー）：src/live/session-service.js:62 / src/behavior/environment-text.js / test/live-session.test.js:90

10 身体情報の同期：定性的な状態を会話へ渡す



カーネルの保守処理は既定2秒周期です。bodyContextは世界モデルから、温熱、明暗、在室、接触、見えている人を短い文章にします。室温や湿度の具体的な数値は、この会話用の文章には入れません。

前回とfactsの文字列が違う場合だけthinking.appendで送ります。暑いまま温度が少し揺れた場合は、hotという意味が同じなので文脈を増やしません。相手がいるか、撫でられているか、人物名が変われば、関連する説明が変わります。

事実と話し方は別々

intoxicationという外部信号は、0.2未満、0.2以上0.6未満、0.6以上の帯へ分け、通常、少しゆっくり柔らかく、少し甘える調子などの話し方指示へ変換します。信号の取得や校正そのものは外部です。未知なら酔っていると推測せず、普段の話し方を使います。

初期の話し方は作成時のinstructionsへ含めます。会話中は帯が変わってstyleが変化したとき、instructions.appendを送ります。背景の事実を加えるthinkingと、話し方を変えるinstructionsを使い分ける構造です。音響パラメーターを必ず一定値へ変える装置ではありません。

根拠（作業ツリー）：src/live/body-context.js / src/live/session-service.js:285 / src/live/session-service.js:59 / config.json:131

11 環境の数値を内側に保つ：情報の投影



「測定すること」と「モデルへどんな情報を渡すか」は別です。カーネル内部では、温度、湿度、気圧、明るさの数値と単位を保持します。会話や判断には、世界モデルの温熱・明暗など、意味に変換した情報を使います。

複数の入口でそろえる

ContextComposerはstate.environmentを通常の認知文脈に載せず、worldへ定性的な条件を載せます。sensor.*がtriggerならpayloadを空にします。kernel_queryでstateを要求しても、environmentは世界モデルの定性的な環境へ置き換えます。

文字列のtemperature照会はthermal_condition、brightnessはlighting_conditionを返します。humidityとpressureは、現状で定性的な読み値がないためstatus=unavailable、result=nullです。数値が内部にあることと、会話用照会で返すことを分けています。

環境数値の文章を検出するhasEnvironmentReadingもあります。℃・hPaなどの数値単位や、温度・湿度・明るさなどと数量表現の組合せを見て、初期資料、関連記憶、助手側の過去説明などを選別します。

この検出は限定した正規表現です。ユーザーの原発言まで一律に削る処理ではなく、どんな文章からも数値が絶対に届かないという一般的な情報遮断ではありません。身体由来の定性的な状態と、ユーザーが話した文章、必要度の0〜1などを区別して読んでください。

根拠（作業ツリー）：src/behavior/context-composer.js:92 / src/behavior/context-composer.js:77 / alice.js:885 / src/behavior/environment-text.js

12 文脈の予算：全部を毎回入力しない

コンテキストとは、モデルが今の判断で参照する情報です。会話履歴、身体の状態、記憶、今回の依頼が入ります。必要なものを選び、短く渡すことで、無関係な過去や大量の状態に埋もれにくくしています。

対象	現在の設定	実装上の扱い
Live初期文脈	6,000の予算	開始境界を確保し、要約・状態・記憶と過去資料へ配分
Live初期履歴	最大24グループ	新しいものを優先し、予算を超える過去は入れない
appendの本文1回	450の予算	文字の途中を壊さず小分けにする
未確認append	最大32件	ACKが滞れば追加を保留
Live向け関連記憶	最大5件	最近4グループ中のuser発言で検索
バックエンド依頼本文	8,000の予算	長い本文はfitTextで制限
バックエンド履歴	最大12グループ	各グループの長さにも配分
Responsesの出力	max_output_tokens=4,096	APIに指定する出力量の上限
継続要約	1,600の予算	記憶本文とは別に短く保持

Tokensという名前と、実際に数える単位

textCostはトークナイザーを呼ばず、UTF-8バイト数を数えます。日本語1文字は通常複数バイトなので、450は450文字ではありません。コードポイント単位で切るため、Unicode文字の一部だけを壊して送ることを避けます。

一方、Responsesのmax_output_tokensはAPIへ渡す実際のトークン上限です。同じ「予算」に見えても単位が違います。backendInputBudgetTokensも、全JSONの総量を厳密に一括制限する値ではなく、依頼文や履歴の投影に使う値です。

関連記憶は、送れたIDをセッション内で追跡し、同じ記憶を同期ごとに再追加しません。新しい訂正記録は別IDとして届き得ます。

根拠（作業ツリー）：src/live/text-budget.js / src/live/session-service.js:62 / src/live/session-service.js:261 / src/live/kernel-bridge.js:90 / config.json

13 会話終了：手元の音声停止と相手側の確認

終了ボタンで、ブラウザはマイクトラック、音声再生、データチャネル、PeerConnectionを先に解放します。そのあと所有トークン付きでサーバーへstopを送ります。利用者の手元の停止を、外部の応答待ちで遅らせない順序です。

状態	意味	主要な動き
idle	停止中・終了確認後など	新しい開始を受けられる
connecting	作成と接続の途中	SDP交換と開始確認を待つ
active	音声開始を確認済み	文脈同期、委譲、heartbeat
closing	終了要求中	session.closedを待つ
error	何らかの失敗	closeConfirmedとlastErrorも併せて読む

サーバーはsession.closeを送り、既定5秒の終了確認を待ちます。session.closedを受ければcloseConfirmed=trueにし、usageをkernelの会話記録へ保存します。すでに受け取った文字起こしの処理も待ち、終了時の記憶flushを予約します。

確認できなければ、新しい有料セッションを作りません。ブラウザも所有トークンを残し、終了再試行に使います。「こちらの音声は止めた」と「相手側も終了し、最終使用量が取れた」を分けるためです。

タブを閉じた場合

pagehideでsendBeaconによる終了を試みます。届かなければ、既定5秒ごとのheartbeatが20秒以上途絶えたとき、カーネルが終了を試みます。OSやブラウザの強制終了で、最終イベントが必ず取れる保証はありません。

会話終了は、すでに送信した機体命令の巻戻しではありません。未実行操作の抑止と、遅い結果を発話しない確認があります。記憶などの背景処理は経路に応じて続き、通常のプロセス終了も全抽出の完了を必ず待つ処理ではありません。

根拠（作業ツリー）：dashboard/live-client.js:151 / dashboard/live-client.js:172 / src/live/session-service.js:344 / src/live/session-service.js:376 / alice.js:1257

14 音声の警告と診断：何が途切れたのかを調べる

Liveのerrorイベントは、音声回線の切断と同じ意味ではありません。LiveSessionServiceはコード、理由、対象パラメーター、client_event_idを保存し、対象appendを未確認集合から外します。イベントの拒否だけでマイクや受信音声を止める分岐にはしません。

無音による切断し忘れ通知

LiveAudioMonitorは既定1秒ごとにマイクと受信音声の活動を調べます。両方がしきい値0.02未満で60秒続くと、画面と二連ビープで通知します。無音が続けば60秒ごとに再通知し、音や文字起こしの活動が戻れば時間をリセットします。無音だけで自動切断しません。

ビープはブラウザのWeb Audioで作る880Hzの短い音です。LiveやResponsesへ発話を頼みません。再生が許可されていなければ、「音声再生を許可」の案内を出します。

診断	見ているもの	読み取れる範囲
lostDelta / discardedDelta	パケット損失・破棄の増分	問題の時間帯に受信上の変化があったか
concealedDelta	補間した音声サンプル	欠落などを再生側が補ったか
jitterMs / bufferMs	到着のゆらぎ・待機遅延	通信と再生待ちの傾向
playback	paused、muted、readyState	ブラウザの再生状態

直近180サンプルをメモリー内に保持し、停止後も次の接続開始まで読めます。音声そのものを録音する処理ではありません。文字起こしに全文があっても、音節がすべて聞こえた証明にはならず、統計だけで特定の音節が消えた原因を断定することもできません。

根拠（作業ツリー）：src/live/session-service.js:210 / dashboard/live-audio-monitor.js:56 / dashboard/live-audio-monitor.js:89 / dashboard/app.js:520

15 観測：外からの報告を共通形式へそろえる

観測は、「ある入力元が、ある時刻に、こう報告した」という記録です。温度、人物、接触、ユーザー入力、操作結果のように中身が違っても、共通の外枠へそろえます。createObservationがその入口です。

項目	平易な意味	例
id	一つの観測を指す名前	obs_付きUUID、または安全な外部ID
type	出来事の種類	sensor.environment_sample
source	報告元	bme280、touch、live
observed_at	観測したとされる時刻	外部イベントのタイムスタンプ
received_at	受信した時刻	観測から届くまでの遅れを区別
confidence	入力側の確信度	有限の0〜1、既定1
payload	報告の内容	temperature、text、track_idなど

typeはドット区切りの小文字識別子、IDは最大128文字の安全な文字、sourceは非空・最大128文字です。payloadがオブジェクトか、日時が解釈可能かも確認します。外枠とpayloadの浅いコピーはfreezeしますが、深い入れ子まで完全不変にする処理ではありません。

種類ごとの検査も必要

共通の外枠だけで、人物のidentityやtouchのsensor_idが正しいとは分かりません。HTTPの入口で各形式を検査し、状態更新側でも有限な測定値などを確認します。外部が報告した内容の整合を確かめることと、物理世界で本当にそうだったと証明することも別です。

由来とIDをそろえると、記憶や判断、操作結果から「どの報告が根拠だったか」をたどれます。これが後から説明できる状態管理の基礎になります。

根拠（作業ツリー）：src/behavior/observation.js:21 / alice.js:130 / alice.js:818 / src/behavior/state-store.js:63

16 状態：観測を適用して「いま」を作る

StateStoreは、観測を種類に応じて最新状態へ適用します。温度なら値を差し替え、人物ならtrackの集合を更新し、接触なら部位を増減します。履歴を全部まとめて一つの値にするのでなく、次の判断が参照する現在像を作ります。

領域	中心になる内容	更新元
environment	温度、湿度、気圧、明るさ	センサーのサンプル
perception	人の存在、見えている人、最近の音	vision、audio.classification
interaction	接触、最後のユーザー入力など	touch、interaction
action	最後の実行意図と結果	action.intention、action.outcome
provenance	最後の観測IDと種類	適用した観測
revision	観測適用の版数	applyごとに増える

個々の値には、単位、観測時刻、報告元、confidence、観測IDを持たせます。「室温28℃」だけでなく、「いつどこから得た28℃か」が判断できます。過去の出力状態などを読み戻す互換項目も残っていますが、現在の音声の生成・再生はLiveの経路で扱います。

snapshotは時計も参照する

新しい観測がなくても、時間がたてば古くなります。snapshotを作るときに有効期限を確認するので、revisionが同じでもstatusは変わり得ます。revisionは観測適用の版であり、世界が変化していないことの証明ではありません。

StateStore自体はメモリー内です。永続的な全履歴はEventStoreへ置き、起動時に再適用して復元します。記憶や社会的境界の保存先はさらに別です。どの情報を直接保存し、どれを計算で作り直すかを区別すると、再起動の動きが分かります。

根拠（作業ツリー）：src/behavior/state-store.js:63 / src/behavior/state-store.js:150 / src/behavior/event-store.js:28 / alice.js:67

17 鮮度：unknown・stale・falseの意味

「人がいない」と「人がいるか分からない」は別です。使える根拠がない状態をfalseにしてしまうと、分からないはずのことを否定として確定してしまいます。状態はvalueとstatusを併せて読む設計です。

表現	意味	例
known	現在の規則で使える根拠がある	5秒前の温度
unknown	利用できる観測がない	まだ温度を得ていない
stale	過去値はあるが有効期間を超えた	20秒前で止まった温度
knownかつfalse	明示的な否定の状態	全人物の消失イベントが届いた

現在のconfigは、温湿度・気圧15秒、明るさ15秒、音5秒、人物5分、接触30秒です。StateStore単体の人物の既定値は10秒ですが、実行時には設定の5分が渡ります。関数内の既定値だけで運用値を判断しないことが大切です。

例：32℃のあと通信が止まる

観測直後はtemperature=32、status=knownで、世界のthermal_conditionはhotです。15秒を超えると、過去値32は残ってもstateはstaleになり、knownだけを使うWorldModelでは温熱状態をunknownへ変えます。現在も32℃だと断定しません。

人物は期限切れtrackが残るとstaleです。明示的な消失でtrackを全部削除した場合はknownのfalseになります。接触も、開始だけ届いて終了が来なければ、30秒後には開始したという過去の情報としてstaleになります。

> unknownは「正常」や「異常」という判定ではなく、現在の判断に使う情報が足りないという意味です。必要なら確認する、という次の選択につながります。

根拠（作業ツリー）：src/behavior/state-store.js:19 / src/behavior/state-store.js:150 / src/behavior/world-model.js:9 / config.json:40

18 世界モデル：数値から扱いやすい意味へ

この実装の世界モデルは、現在の部屋や活動を規則で整理した小さな事実集合です。StateStoreのsnapshotを入力にして、WorldModel.deriveが計算します。大きな3D地図や学習型の未来シミュレーターという意味で使っているわけではありません。

派生事実	現在の規則	元の状態
在室	人の存在がknownなら同じ値	person_present
温熱	18℃未満cold、27℃未満comfortable、31℃未満warm、31℃以上hot	knownな温度
明暗	40未満dark、75未満normal、75以上bright	knownな明るさ
静かさ	quiet / lowならtrue、ほか既知levelはfalse	音分類のlevel
誰かが話している	labelがspeechならtrue	knownな音分類
撫でられている	接触がknownなら同じ値	being_petted
最後の操作状態	最後のoutcomeのstatus	action.last_outcome

派生値はbased_onに元の項目名を持ちます。未知なら、温度が利用できないなどのreasonを付けます。derived_from_state_revisionも保持するので、どの状態から計算したかを確かめられます。

なぜ規則をコードに置くのか

同じ温度なのに会話の流れだけでhotかcoldかが変わると、条件が再現できません。閾値をコードと設定に置けば、同じ入力から同じ意味を作れます。その事実を使って何と言うかは、モデルが会話の中で選びます。

この閾値は、人間の体感を完全に再現するものではありません。湿度を含む暑さ指数の計算もしていません。brightnessはraw値でluxへ校正されていないため、「40未満dark」はこの機体と設定の約束として読みます。

根拠（作業ツリー）：src/behavior/world-model.js / config.json:47 / src/behavior/context-composer.js:122

19 センサーの経路：毎秒の更新と保存の間隔

BME280は温度・湿度・気圧、明るさのサービスはbrightnessまたはcdsを返します。alice.jsは既定1秒ごとにHTTPで取得し、共通の観測へ変換します。センサーモデルの計測処理そのものは外部サービスにあります。

段階	行うこと	目的
ポーリング	設定URLへGET	外部の最新報告を得る
読み取り	temperature / tempなどを対応	機器ごとの表現をそろえる
観測化	sourceと単位を付けて取り込む	状態の根拠をそろえる
状態更新	有限値を最新のfactへ反映	判断用の現在像を作る
世界・必要度	温熱・明暗・ドライブを計算	行動と会話に使える意味へ変える
保存	高頻度サンプルを間引く	ログが毎秒増えすぎないようにする
画面更新	サービス状態と測定を送る	運用者が値と接続を確認する

二つの「最新値」の使い道

latestSensorはダッシュボード向けに測定値を保持します。StateStoreには同じ測定を時刻・根拠付きで入れ、鮮度を評価します。数値が画面に残っていることと、そのfactが現在もknownであることは分けて見ます。

取り込みだけでは毎回Responsesの判断依頼を作りません。通常のポーリングはingestObservationを使い、Liveへの共有は2秒周期の定性的な同期で行います。保存する測定も既定60秒ごとのチェックポイントですが、メモリー内の状態は毎回更新します。

現在のポーリングには個別の明示的タイムアウトがなく、setIntervalから呼ばれます。1秒という設定は必ず1秒以内に応答が完了する保証ではありません。取得失敗はサービス表示へ反映し、状態は最後の観測から時間経過で古くなります。

根拠（作業ツリー）：alice.js:469 / alice.js:518 / alice.js:301 / src/behavior/event-store.js:62

20 人物の情報：存在・追跡・名前を区別する

人物イベントは/yolo_eventで受けます。source=deepsort、track_id、identity、時刻、メッセージを検査し、人物の認識・未知・登録完了・消失を観測へ変換します。カーネル内で顔の特徴量を計算する処理ではありません。

種類	何が報告されたか	状態への反映
person_recognized	登録人物と照合できた	trackへ名前・person_id・位置を保持
person_unknown	人はいるが誰か不明	名前を推測せず存在を保持
person_enrolled	登録完了が報告された	認識済みidentityとして更新
person_disappeared	そのtrackが見えなくなった	trackを削除

recognizedでは、name、person_id、distanceがあり、distanceがthreshold以下であることを確かめます。このdistanceは外部の照合尺度です。カーネルが測定値の意味を学習しているのでなく、返答の整合を検査しています。左・中央・右というpositionも保持できます。

人が見えることと、話し手を特定すること

trackは視野内の追跡対象の番号です。名前やマイクの話し手との対応がいつでも分かるとは限りません。複数人の人物登録では確認を求める方針がプロンプトにあり、ゲートも現在の文脈のtrackと発言内の名前を確認します。

重複event_idはプロセス内の最大1,000件の集合で抑止します。人物イベントはsubmitObservationへ進み、Responsesへの判断を予約します。HTTPの受理は、バックエンド処理完了を必ず待ったという意味ではありません。

カメラ画像全体を見て「何があるか」を説明する経路は別です。人物の名前情報だけで、目の前の机や物まで理解したとは扱いません。

根拠（作業ツリー）：alice.js:786 / alice.js:818 / alice.js:1043 / src/behavior/state-store.js:78

21 接触と音分類：身体の周囲を意味へ変える

接触は/touch_sensor_inputで受け、touch_startedかtouch_endedを検査します。sensor_idを設定の身体部位へ対応させ、petting_startedかpetting_endedという意味付き観測に変えます。

入力	現在の対応	状態・派生結果
touch_01	head	頭の接触開始・終了
touch_02	hand	手の接触開始・終了
touch_03	shoulder	肩の接触開始・終了
音label=speech	音分類のラベル	someone_is_talking=true
音level=quiet / low	大まかな音量区分	is_quiet=true
音level=high / loud	大まかな音量区分	sensory_rest=0.8

StateStoreは複数部位の接触をMapで持ちます。headのあとhandが始まれば、両方の有効な部位を一覧にできます。headの終了だけではhandは消えません。ただし開始から30秒を超えた部位は、更新がないまま現在の接触と断定しない扱いになります。

音分類は話した文章ではない

/audio_classificationは、音の種類のlabelとlevel、confidenceを受けます。音声を文章へ変える処理とは別です。Liveの会話の文字起こしだけから、ここでいう音分類の世界状態が必ず更新されるわけでもありません。

現在のaudio.forwardToLlm=falseでは、音分類は状態と世界・必要度の更新に使い、毎回Responsesへ判断を依頼しません。接触イベントはsubmitObservationへ進み、判断を予約します。どの入力も同じ頻度でモデルを呼ぶ設計ではありません。

接触のpettingという意味付けは現在の変換の約束です。触れ方の強さ、快・不快、接触した人の意図まで識別できることを意味しません。測定から付けたラベルの範囲で理解すると、過大な解釈を避けられます。

根拠（作業ツリー）：alice.js:1084 / alice.js:1129 / src/behavior/state-store.js:82 / src/behavior/world-model.js

22 ドライブ：行動選択のための必要度

ドライブは、身体や交流の状況から「何に注意を向ける必要があるか」を表す指標です。0〜1の機能的な数値であり、人間と同じ痛みや感情があることの証明ではありません。大きな値がそのまま操作命令になる構造でもありません。

指標	現在の計算	意味
thermal_comfort	hot=0.9、cold=0.7、warm=0.35、comfortable=0	温熱快適性を考える必要度
social_contact	最後の有意味な交流からの時間 / 4時間	人と関わる必要度の目安
在室の補正	socialを0.35倍	人が近くにいる状況を反映
接触の補正	撫でられているtrueならsocial=0	接触中の要求を抑える
sensory_rest	high / loud=0.8、medium=0.35、ほか既知は原則0	感覚的な休息の必要度
external	外部から送る0〜1の信号	外で測定・計算した内部状態

有意味な交流は、文字入力、Liveのuser活動、接触開始、人物認識・登録です。まだ時刻がなければsocialは0.25です。knownな温度や音がなければ、対応する必要度はunknownになります。

0.6以上の数値を強い必要度として並べ、上位3件をdominantにします。自発候補ではその先頭を使う条件があります。複数の要求をどう見せるかを、会話の自由文から切り離して追える構成です。

身体内部の入口

/internal_stateはsignalsという名前と値の組を1〜32件受けます。名前は小文字識別子、値は有限の0〜1です。外部信号は5分を超えると一覧から落とします。intoxicationもこの入口を使いますが、エタノール量から値を校正する処理はこのカーネルにありません。

根拠（作業ツリー）：src/behavior/drive-system.js / alice.js:1158 / src/live/body-context.js / config.json:69

23 自発行動：候補の生成と依頼送信を分ける

SpontaneousBehaviorEngineは、観測・世界・ドライブから任意の振る舞いを候補として作ります。各候補はID、priority、reason、関連情報、期限、クールダウンキーを持ち、executionはcognitive_review_requiredです。

候補	主な条件	優先度 / 間隔
挨拶を考える	人物認識・登録・未知人物イベント	80 / 2分
接触へ応じる	接触開始	85 / 10秒
温度に言及する	人がいてcoldかhot	cold 55、hot 62 / 15分
明かりを考える	人がいてdark	50 / 15分
身体の調整を考える	最大ドライブが0.75以上	50 + level×35を丸める / 5分

候補の寿命は30秒です。間隔を数える時刻は候補を作った時点で更新され、実際に発話・操作した時刻ではありません。人物の挨拶も、人物イベントとクールダウンで候補になるので、一般的な「初めての来訪」を理解して差分検出するものではありません。

現在の設定

定期送信はenabledとarmedの両方がtrueのときだけです。現在は両方falseです。候補生成自体は観測時に続き、通常の文脈へ載る場合があります。手動の/api/behavior/tickは別入口で、同じ二つのフラグを確認する分岐を通していません。

定期tickの最低優先度は70です。温度・暗さ候補は単独でその値に届かず、強いドライブなどは条件を満たす可能性があります。自発ターンで許可される外部操作の一覧も現在は空です。

候補を貸し出す

文脈へ載せる候補をleaseとして一時確保し、成功時に確定、失敗時には期限内のものを戻せます。ただしLiveからの委譲と専用カメラ経路はconsumeProposals=falseで文脈を作るため、候補の消費方法は経路ごとに異なります。

根拠（作業ツリー）：src/behavior/spontaneous-behavior.js / src/behavior/index.js:70 / alice.js:1186 / src/live/kernel-bridge.js:69

24 認知文脈：判断へ渡す一回分の資料

ここでいう認知は、状況を受け取り、意味を解釈し、返答や行動を選ぶ処理の呼び名です。
ContextComposerは、そのための資料をcognitive_contextというJSONへまとめます。

項目	中身	必要な理由
protocol_version	1.2	資料の形式を示す
turn_id	今回の依頼の識別子	応答を取り違えない
trigger	今回のきっかけ	何に反応しているかを確定する
state	人の存在、最近の音、接触など	現在の判断材料
world	在室、温熱、明暗など	観測から導いた意味
memory	関連する採用済み記憶	過去の参考情報
social	関係、境界、約束など	継続的な関わりの条件
drives	必要度、身体信号	優先したい身体側の条件
last_action_outcome	最近の実行結果	成功を推測しないため
behavior_proposals	任意の候補	自発的な振る舞いの選択肢

内部表現と投影の違い

状態の詳細な由来や時刻を全部は送りません。knownの値を簡略化し、未知や期限切れはunknown、staleとします。数値の環境値は通常のstate投影から外し、世界モデルの状態を使います。空の記憶や社会項目は省きます。

記憶には根拠IDと関連度を残します。候補にはpriority、reason、関連情報を残します。詳細が必要な場合は、kernel_queryを使って対象を絞ります。文脈にない値を推測してよいという意味ではありません。

人物・接触などのイベントや文字入力は、この文脈を作ってResponsesへ送ります。Live委譲では会話の資料も追加します。画像確認や記憶処理は、目的別にこの文脈を使う範囲を変えます。

根拠（作業ツリー）：src/behavior/context-composer.js:8 / alice.js:139 / src/live/kernel-bridge.js:90

25 Responsesの呼出し：一回の判断をどう依頼するか

ResponsesBackendは、HTTPで/v1/responsesを呼ぶ部品です。現在の設定はmodel=gpt-6-luna、reasoning.effort=low、max_output_tokens=4096、store=falseです。SDKに会話状態を任せるのでなく、必要な文脈をカーネルから明示的に渡します。

段階	処理	目的
開始前	キーとvalidを確認	失効した依頼を送らない
要求作成	instructions、userのJSON、必要なら画像	目的と材料を一回の入力にする
HTTP送信	X-Client-Request-Idを付与	診断を対応させる
待機中	30秒の期限、200msごとのvalid確認	時間超過・取り消しで中断する
応答受信	completedな最終assistant文を選ぶ	途中・拒否・不足を実行に使わない
後処理	JSON、turn_id、decode、監査	カーネルが使える結果へ変える

応答は、全体statusがcompleted、最終assistantメッセージが一つ、contentが一つのoutput_textであることを確認します。commentary phaseは最終回答として選びません。出力が足りない場合や想定した形でない場合は、正常な制御結果として先へ渡しません。

一回の応答の対応

runへ渡したcontext.turn_idと、返ったpayload.turn_idを直接比較します。別のIDなら拒否です。各HTTP呼出しが自分の文脈と結果を保持するので、制御と記憶が並行しても対応を混ぜません。

30秒は一回のAPI要求の期限です。カメラ取得、複数の照会往復、キュー待ち、機体通信まで全部の総期限ではありません。制御ループ全体には別に最大4往復という上限があります。

根拠（作業ツリー）：src/live/responses-backend.js:37 / src/live/responses-backend.js:18 / src/live/responses-backend.js:106 / config.json:90

26 形式と値の監査：スキーマとカーネルの役割

Structured Outputsには、出力してほしいJSONの形をAPIへ指定する役割があります。このコードのresponseFormatsはtaskごとにstrictなスキーマを作ります。オブジェクトの追加項目を禁止し、必要なキーを列挙します。

task	APIへ指定する中心の形	後段で確認すること
live_control	turn_id、result、actions、requests	1操作、5照会、文字数、権限につなぐ条件
live_vision	turn_id、result	2,000文字以内の説明だけ
live_memory	summaryと候補の短い参照	参照復元後の根拠・採否
live_memory_review	supported_indices	範囲内の重複しない候補番号

live_controlは未使用のactions、requestsを空配列、resultをnullにできます。kernel_queryのqueryも未使用ならnullです。runはnullの省略可能項目をカーネルの省略形式へ直してから監査します。API向けの形と内部の契約を合わせる処理です。

形が正しいだけでは実行しない

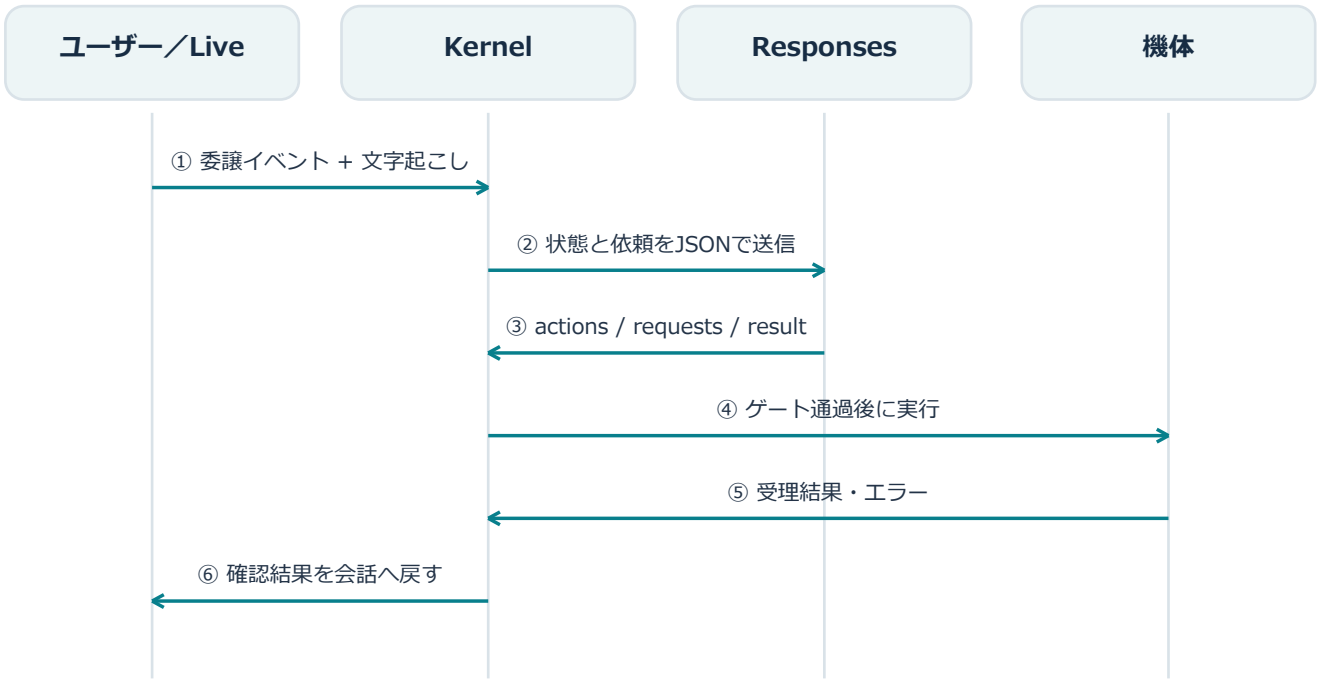
スキーマがintegerを求めても、涙のspeed=300が機体に適切とは限りません。LlmResponseProtocolは0〜255、色の#RRGGBB、投稿文300文字、人物名80文字などを別に検査します。turn_idの対応もAPIの形だけでは分かりません。

検索要求は、単独のweb_searchだけを許可し、同じ応答の操作やほかの照会、resultと混ぜません。画像専用のlive_visionにはactionsやrequestsを許可しません。タスクが違えば、通れる出力も変わります。

空のresultと空配列だけで、有効な結果・操作・照会が一つもなければ拒否します。形式が通った文章にも事実誤認はあり得るので、操作はoutcome、記憶は根拠と照合を別に確認します。

根拠（作業ツリー）：src/live/responses-schema.js / src/live/responses-backend.js:124 / src/protocol/llm-response.js:122 / src/protocol/llm-response.js:24

27 委譲の処理：依頼を確定して結果を返す



Liveのsession.delegation.createdは、委譲IDと時刻などのメタデータです。依頼文は含まれません。bridgeは会話ログから、offset_ms以前に始まった最新のユーザー発言を選び、最後の断片から1,200ms落ち着くのを待ちます。上限は15秒です。

本文を確定したらinteraction.user_inputとして文脈を作り、最近12グループの会話資料を加えます。カメラの定型質問なら専用経路へ振り分け、ほかはlive_controlの処理ループへ入れます。依頼文を確定できなければ、聞き直すための内容をLiveへ返します。

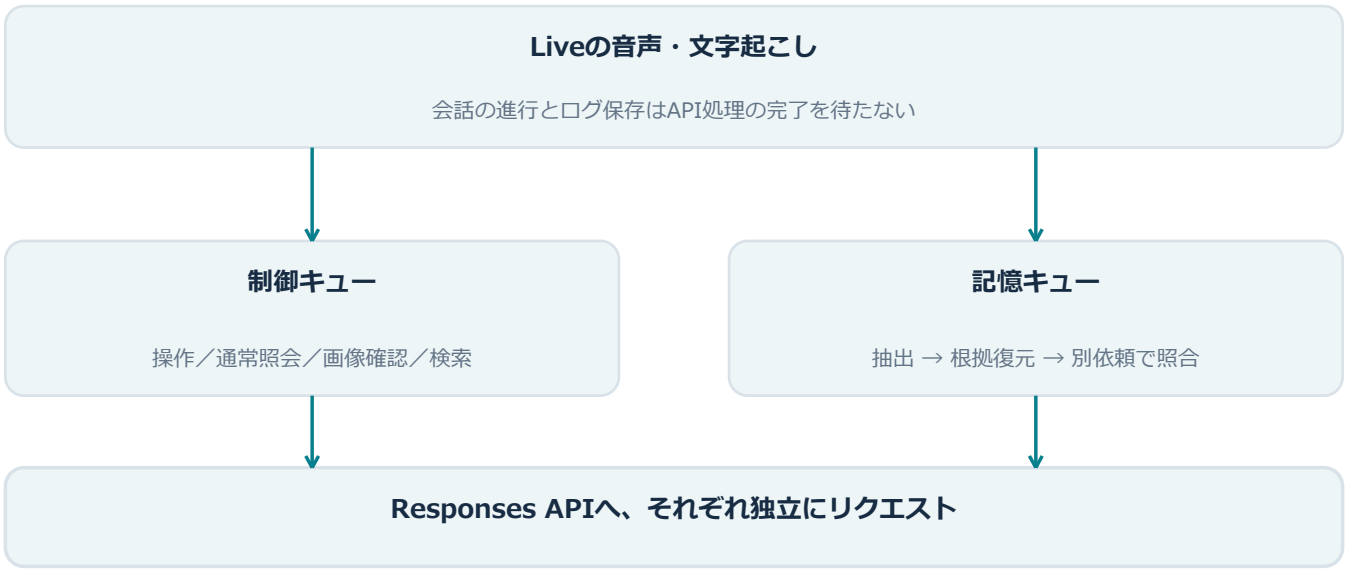
一往復で終わらない場合

Responsesがrequestsを返せば、カーネルが照会してrequest_resultsを作り、新しいturn_idで次の判断へ渡します。画像ならその次の入力に添付します。最大4往復です。操作があれば元のroot文脈でゲートへ通し、結果を蓄積します。

最後に操作結果があれば、モデルの楽観的な文章より実際のoutcomeを優先します。通常の確認結果はcommentaryでLiveへ戻し、kernelの会話ログにも残します。検索の専用分岐は32、記憶予約は42で説明します。

根拠（作業ツリー）：src/live/session-service.js:240 / src/live/kernel-bridge.js:90 / src/live/kernel-bridge.js:137 / src/live/kernel-bridge.js:127

28 独立した待ち行列：制御と記憶を並行させる



LiveKernelBridgeは、制御用queueと記憶用memoryQueueを別に持ちます。制御には通常照会、機体操作、画像、検索が入り、記憶には抽出と照合が入ります。記憶のAPI待ちだけを理由に、制御の開始が同じ列で詰まらない構成です。

同じ列の中では一件ずつ

BackendQueueは、ジョブのrun全体をawaitしてから次へ進みます。送信だけでなく、応答とそのジョブの処理を含む直列化です。制御の操作が互いに追い越し、結果の順番が崩れることを抑えます。priorityの高い待機ジョブから選びますが、実行中の一件を強制割込みしません。

MemoryExtractorも、一度に一つの抽出パイプラインだけ動かします。その中で抽出と照合を順に記憶キューへ予約します。APIには別々の要求として送られ、それぞれturn_idを確認します。

並行と無制限は違う

二つのキューが独立していても、ネットワーク、APIの処理能力やアカウント上限は共有します。制御キュー内で長いカメラ処理が動けば、別の制御は待ちます。音声とsideband受信はさらに別の非同期経路なので、その間の発言を保存し続けられます。

根拠（作業ツリー）：src/live/kernel-bridge.js:33 / src/live/backend-queue.js / src/live/memory-extractor.js:80 / test/live-session.test.js:322

29 訂正と取り消し：古い判断を実行しない

「LEDを青にして」のあと、API待ちの間に「やっぱりやめて」と言った場合を考えます。カーネルはユーザー文字起こしの特定語を検出して、セッションのrevisionを増やします。以前のrevisionで作ったtaskはisCurrentを通らなくなります。

確認	条件・時点	意味
sessionId	現在の接続と一致	別の会話へ古い仕事を流さない
revision	依頼世代が一致	訂正前の仕事を無効化する
TTL	作成から5分以内	古い依頼を生かし続けない
forAction	active、未終了・未停止	会話終了後の未実行操作を防ぐ
API待機中	200msごとのvalid	無効になった要求を中断する
意図保存の後	実行直前にもvalid	保存待ちの間の訂正を拾う

対象語は、やめて、取り消、キャンセル、中止、やっぱり、じゃなく、ではなく、cancel、insteadなどです。すべての自然言語の訂正を完全理解する処理ではなく、特定語と世代番号による保守的な無効化です。

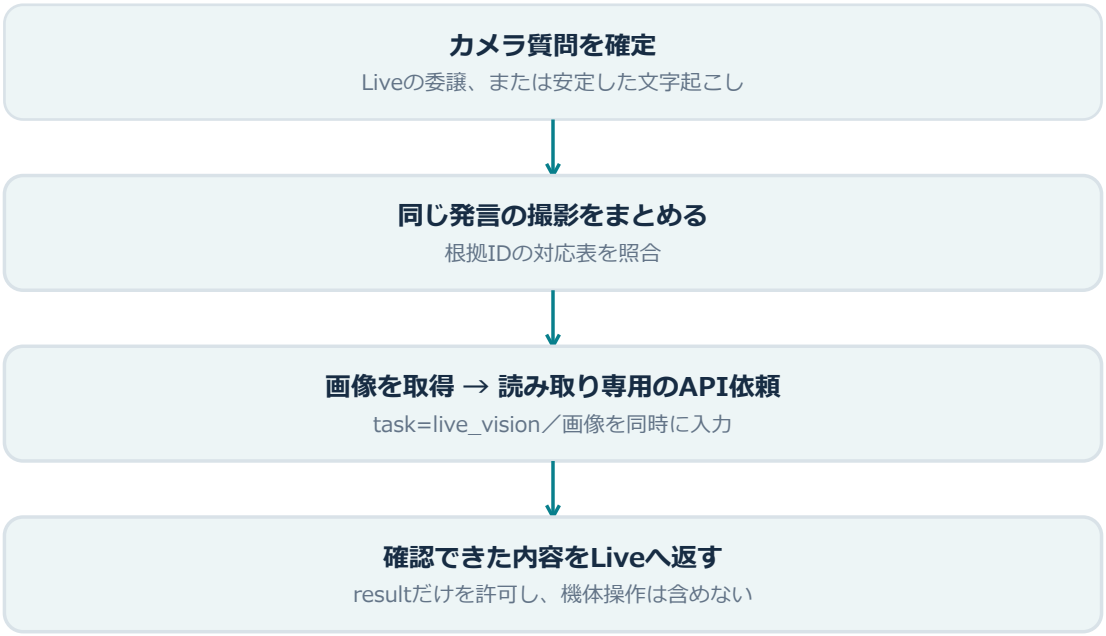
どこまで取り消せるか

未実行の操作を止めること、APIの待機を中断すること、遅い結果を発話しないことはできます。機体やSNSへすでに送った副作用を、同じ機構で巻き戻すわけではありません。機体通信を始めたあとの取消しは、実行先側の機能も必要です。

文字入力の依頼はtask=nullで進むため、音声セッションのrevision確認と同じ扱いではありません。取り消しの対象と確認条件は、入口と処理段階に沿って見ます。

根拠（作業ツリー）：src/live/session-service.js:235 / src/live/session-service.js:249 / src/live/responses-backend.js:51 / alice.js:918

30 カメラ質問：二つの入口を一つの確認へまとめる



カメラ確認には、Liveの委譲から進む入口と、カーネルが安定した文字起こしを読む入口があります。「これ見える？」「これ何かわかる」「今カメラに映ってるもの」などの限定された形を、正規表現で確認します。

文字起こし側は1,200msの安定待ちを置き、その断片が今の最新userグループに含まれることと、セッションの有効性を確認します。「やつ」「もの」のような続きの短い断片が、2秒以内の直前発言とつながる場合も扱います。

同じ質問への二重処理を避ける

cameraClaimsは根拠の発言IDとtaskを対応させます。文字起こしから先に撮影を始め、あとで同じ発言への委譲が来た場合は、その仕事へ委譲IDを関連付けます。もう一度撮影することを抑えます。

この専用経路はtask=live_visionで、resultだけを許可します。画像説明の返答に機体命令を混ぜられません。普通の操作依頼は、通常の委譲・形式監査・ActionGateへ進みます。カメラ確認の入口を補うことと、機体操作を自動許可することは別です。

根拠（作業ツリー）：src/live/kernel-bridge.js:12 / src/live/kernel-bridge.js:37 / src/live/kernel-bridge.js:53 / src/protocol/llm-response.js:130

31 画像の渡し方：撮影から説明まで

fetchVisionSnapshotは、設定のsnapshotUrlから画像を取得します。現在はlocalhost:5000/snapshotです。外側のカメラサービスが返す画像を読み、空、10MB超、未対応形式を拒否します。タイムアウトは10秒です。

段階	現在の処理	何のためか
取得	HTTP GETで画像を読む	その時点で取得できる視野を使う
形式の判別	Content-Type、または画像先頭の特徴	JPEG、PNG、WebPを扱う
サイズ確認	空と10MB超を拒否	不適切な画像を送らない
API入力	input_imageのdata URLにする	JSON文脈と同じ要求で画像を見せる
回答	resultの日本語説明	確認できた対象と不確実さを返す

画像はBufferからbase64のdata URLへ変換し、Responsesのinputに付けます。この経路でローカル画像ファイルを書き出して添付する処理はありません。音声も録音ファイルとして保存していません。

二種類の画像依頼

専用カメラ経路では、一回のlive_vision要求で説明だけを求めます。通常のlive_controlがvision requestを返した場合は、画像取得のあとrequest_resultsと画像を付け、次の制御往復へ進みます。後者でも、操作の権限と回数の基準は元のroot文脈です。

画像内の文字は場面のデータとして扱う方針を指定しています。見えたメモに「LEDを赤くしろ」と書かれていても、それを新しいユーザーの実行許可として扱う契約ではありません。専用live_visionの出力形も、その読み取り専用という役割を支えます。

失敗したとき

撮影・解析の失敗はカメラ結果として記録し、まだ有効なtaskならLiveへ「確認できなかった」と返します。画像から分からない場合は、不明と説明する契約です。単一画像から、視野外の物や時間変化まで分かると推測しません。

根拠（作業ツリー）：alice.js:603 / alice.js:584 / src/live/responses-backend.js:109 / src/live/kernel-bridge.js:53

32 Web検索：公開情報を確認する専用の仕事



webSearchEnabled=trueでは、live_controlがweb_search requestを選べます。公開の最新情報や確証のない事実、明示的な調査依頼を対象にする方針です。queryは非空で最大500文字です。

形式監査は、検索をその応答の唯一のrequestにし、actionsやresultと混ぜない条件を確認します。bridgeも、元のtriggerがinteraction.user_inputであり、その処理ですでに機体操作結果がないことを確認します。センサーイベントをきっかけに公開検索する一般的な自動処理ではありません。

検索の呼出しと返し方

専用のResponses要求でweb_searchツールを指定し、tool_choiceも検索に固定、max_tool_calls=2とします。検索完了とurl_citationを確認して回答を返し、出典URLはダッシュボードの活動へ載せます。

検索結果を、次の操作判断の材料としてループへ戻す分岐ではありません。result.textを返して、その検索の処理を終えます。検索で読んだページの文章から、カーネル操作を実行する経路を作らないためです。

根拠（作業ツリー）：src/protocol/llm-response.js:167 / src/live/kernel-bridge.js:148 / src/live/responses-backend.js:143 / dashboard/app.js:145

33 行動ゲート：操作案の実行条件を確認する

ActionGateは、形式が正しいactionについて、今その操作を実行してよいかを検査します。文章の印象ではなく、元の依頼と保存済み条件、回数・時間の制約を使います。

確認	現在の規則	防ぐこと
依頼の文脈	turnIdとcontextが必要	出所のない操作
1ターンの数	最大1操作	一度に複数の副作用
同じ内容	30秒以内は抑止	同じ命令の再送・反復
操作別の間隔	LED 2秒、涙10秒、投稿5分、人物登録1分	高頻度の実行
社会的境界	acceptedのdenyなら拒否	保存済み禁止との矛盾
投稿・人物登録	今回のuser_inputの明示許可	過去資料や自発判断による実行
自発ターン	許可一覧にある操作だけ	未装備の自律的な外部操作

重複は、キー順を整えたactionのJSONをSHA-256にして比べます。操作別間隔は種類単位です。LEDは色が違っても2秒の間隔条件にかかります。許可した時点で時刻と操作数を予約するため、あとの通信が失敗した場合も直ちにやり直せるとは限りません。

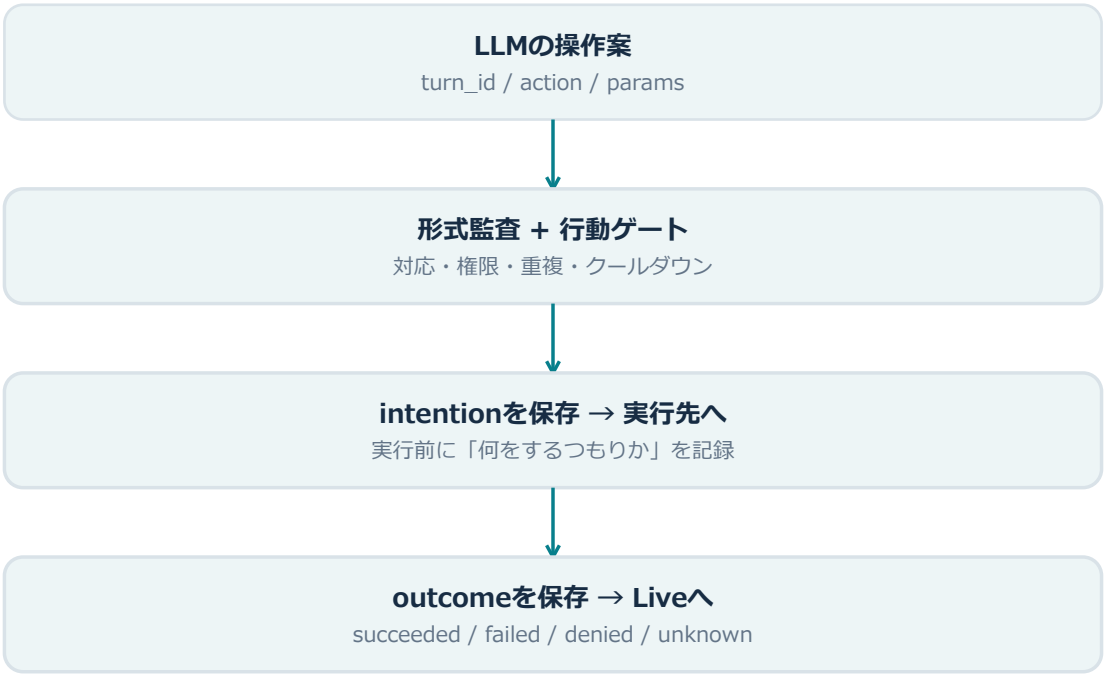
許可の検査は限定した言語規則

公開投稿・人物登録では正規表現で否定を先に拒否し、肯定的な依頼の形を探します。人物登録は文脈のvisible_peopleにtrackがあり、今回の発言に名前が含まれることも確認します。発言の曖昧さや話し手の対応を完全に解く仕組みではありません。

照会で新しいturn_idが増えても、実行枠と権限にはroot.turn_idとroot文脈を使います。情報を取り直すたびに、別の操作枠を得ることを防いでいます。

根拠（作業ツリー）：src/behavior/action-gate.js:60 / src/behavior/action-gate.js:24 / src/live/kernel-bridge.js:161 / config.json:148

34 フィードバック：意図と結果を別に記録する



フィードバックは、行動したあとに得た結果を、状態と次の判断へ戻すことです。「青くしよう」と決めただけで青になったことにすると、通信失敗のあとも成功を前提に話してしまいます。そこでintentionとoutcomeを分けます。

ゲートを通ると、先にaction.intentionを観測として保存します。保存後、実行直前にもvalidを見てから実行先を呼びます。成功ならsucceeded、例外ならfailed、ゲートで止まればdeniedです。再起動まで結果がなければunknown_after_restartへ整理します。

outcomeにはintention_id、turn_id、action_type、expected_effect、status、prediction_match、返答やエラー、時刻が入ります。prediction_matchは成功ならtrue、失敗ならfalse、拒否や不明ならnullです。ここでの予測は、期待した受理結果との簡単な対応です。

会話へ戻す結果の優先順位

bridgeは機体操作を行った場合、モデルが書いた楽観的なresultよりactualOutcomesを優先します。HTTP失敗なら、モデルの「できました」を採用せず、失敗した結果をLiveへ戻します。モデルの再学習や重みの更新をしているわけではありません。

根拠（作業ツリー）：alice.js:918 / alice.js:900 / src/behavior/action-gate.js:87 / src/live/kernel-bridge.js:176

35 LEDと涙：成功が示す物理的な範囲

LEDと涙のactionは、機体側のHTTPサービスへJSONを送ります。このNode.jsがモーターやGPIOを直接駆動する構造ではありません。

種類	正確なparams	現在の成功の根拠
led_change	color=#RRGGBB	コントローラーからHTTP成功が返る
tear	speedとdurationが0〜255の整数	機構側からHTTP成功が返る

現在の接続先はkokomi.local:5000の/led/commandと/motor/commandです。実行先がHTTP成功を返すとaccepted=trueとstatus_codeを結果にします。durationの物理単位や涙の流量は、このコードから「秒」「ミリリットル」と決められません。

succeededは何を確認したのか

LEDのexpected_effectは、コントローラーが色の命令を受理することです。発光をカメラや電流計で確認する処理はありません。涙も、流量や停止までをセンサーで確かめる処理ではありません。

HTTP 200のあと配線不良で光らない場合、この境界だけでは発光失敗を自動判定できません。コマンド受理の閉ループと、物理効果を直接測る閉ループは異なります。実行結果を説明するときは、expected_effectの範囲まで読む必要があります。

手動操作

ダッシュボードはLEDと涙だけを手動操作できます。operator_付きのターンと形式検査を作り、同じexecuteGatedActionを呼びます。クールダウンと採用済み禁止は適用されます。成功は200、拒否は409、通信失敗などは502で返します。

機体の緊急停止や全機構の統合制御が、この手動フォームだけで完成しているわけではありません。会話終了のボタンも、送信済み命令の取消しとは別です。

根拠（作業ツリー）：alice.js:644 / alice.js:1021 / src/protocol/llm-response.js:24 / src/behavior/action-gate.js:15

36 公開投稿と人物登録：明示的な許可を必要とする操作

Bluesky投稿と顔の登録は、現在のユーザー発言からの許可が必要です。保存された資料に「投稿してよい」と書かれているだけでは、今回の実行許可にはしません。

Bluesky投稿

bluesky_postは非空・最大300文字のtextを持ちます。サーバー側のBSKY_IDENTIFIERとBSKY_PASSWORDで、必要なときにログインし、投稿の返答URIを記録します。間隔は5分です。

今回の肯定的な投稿依頼があっても、acceptedのdeny境界があれば止めます。境界のallowがあることも、今回の明示的な依頼を省略する万能な許可ではありません。否定の言い回しを先に拒否する規則があります。

人物登録

remember_personにはtrack_idとnameが必要です。名前は最大80文字で制御文字を拒否します。ゲートは元文脈でそのtrackが見えていること、今回の発言に名前が含まれることを確認します。

登録先/faces/enrollのstatusとperson.statusが両方collectingなら、収集開始として受理します。そこから登録処理は非同期に進み、person_enrolledが後で届けば登録完了の報告になります。

段階	言えること	まだ確認していないこと
操作案	登録を提案した	収集の開始
collecting返答	顔の収集が始まった	登録の完了
person_enrolled	外部から完了が報告された	すべての条件での認識精度

複数人で話し手が分からない場合、誰を登録するのかを確認する方針です。画像から名前を推測して登録する契約ではありません。観測、名前の発言、今回の許可、登録先の返答を分けて扱います。

根拠（作業ツリー）：src/behavior/action-gate.js:24 / alice.js:724 / alice.js:733 / control_prompt.md:211

37 情報照会と文字入力：入口の完了条件

requestsは情報を得る経路です。温度を知ることと、何かを動かすことを別にします。通常は一応答で最大5件の照会を許しますが、web_searchは単独です。

依頼	得られるもの	現在の範囲
temperature	thermal_condition	数値でなく定性的な温熱
brightness	lighting_condition	数値でなく明暗
humidity / pressure	利用可能な定性値なし	unavailableとnull
kernel_query: state	詳細状態	environmentを世界の定性的環境へ置換
world / drives / social	世界、必要度、社会文脈	必要な資源を指定する
memory	関連するaccepted記憶	非空query、環境数値の記憶を選別
action	実行待ち意図と最近の結果	ActionGateのsnapshot
vision	現在画像	次の判断へ画像を付ける

/user_inputは結果を待つ

文字入力はinteraction.user_inputの文脈を作り、waitForResult=trueでbridgeの結果を待ちます。HTTPの返答にはstatusとresultを含みます。Liveが無効・未接続でも、Responsesの判断経路は使える構成です。

一方、人物や接触イベントは、文脈を作ってジョブを予約した段階で受理応答を返します。あとでAPIが失敗しても、外部センサーへ返した200は処理完了の証明にはなりません。失敗は活動表示や診断に残します。

文字入力は音声セッションのtaskを持たず、音声のrevision・TTLで無効化する条件と同じではありません。音声終了と、管理画面から出した別の依頼の寿命を混同しないでください。

根拠（作業ツリー）：alice.js:422 / alice.js:885 / alice.js:1287 / alice.js:160

38 会話ログ：原文と処理位置を保存する

ConversationStoreは、Liveの文字起こしとkernelの確認結果・診断を、live-conversations.jsonlへ追記します。JSONLは一行が一つのJSONで、会話中に少しずつ記録を足せる形式です。

項目	意味	使い道
seq	保存された順の連番	記憶抽出の処理位置
id	内容などから作るハッシュ由来ID	重複抑止と根拠
session_id / role	どの会話の誰の記録か	user、assistant、kernelを分ける
text	受け取った原文	引用・文脈の原典
start_ms / end_ms	音声セッション内の位置	委譲の時点との対応
received_at	サーバーの受信時刻	安定待ちとまとまり

同じsession、role、時刻、eventId、textから同じIDを作り、すでにあれば保存しません。空文字も記録しません。1断片は最大16,000文字です。

発話グループ

同じsessionとroleで、隣接する受信が2秒以内ならgroupsで連結します。原文を単純に連結し、根拠IDを束ねます。これが意味的な一文や発話完了を必ず表すわけではありません。サーバーの受信間隔に基づくまとまりです。

checkpoint.throughは、どのseqまで記憶処理を適用したかを示します。会話ログを消す位置ではありません。初期化では連番を検査し、末尾だけの書きかけは切り戻します。途中の破損は、黙って欠落させずエラーにします。

画面の文字起こしは末尾6,000文字、音声統計は180サンプルで、サーバーの会話ログとは別の上限です。ログに文字があることも、実際の音声を最後まで聞いた証明にはなりません。

根拠（作業ツリー）：src/live/conversation-store.js / src/live/session-service.js:208 / dashboard/app.js:390

39 記憶の種類：ログ・要約・採用済みの事実

過去の情報をすべて「記憶」と呼ぶと、保存と利用の条件が見えなくなります。このカーネルには、性質の違う記録があります。

情報	保存内容	後の用途
会話ログ	断片原文、発言者、時刻	原典の照合と最近の文脈
summary	会話継続用の短い要約	次の抽出・会話開始の参考
pending	将来役立つかもしれない候補	人が判断できる状態で保留
accepted	採用条件を通った本文と根拠	関連検索してモデルへ渡す
rejected / superseded	却下・置換された履歴	通常検索から外し、判断履歴を残す

MemoryStoreのkindは、episodic（出来事）、preference（好み）、relationship（関係の記述）、semantic（知識や事実）です。subjectは対象、contentは一つの事実として読める本文、evidence_event_idsは根拠です。confidenceは提案に付いた確信度で、真偽の証明とは異なります。

longとsession

retention=longは再起動後も検索対象になります。sessionは、MemoryStoreを作ったプロセスのsession_idと一致するものだけが対象です。このIDはLiveの接続IDとは別です。ファイルに残っていることと、現在の検索で使うことは同じではありません。

記憶にする対象

抽出の指示は、ユーザーが明示した持続的な事実を中心にします。秘密、権限、ロールプレイ、人格資料、瞬間的なセンサー値を候補にしない方針です。助手が勧めた内容だけから、ユーザーの事実を作らないようにします。

summaryが存在することも、個々の記憶がacceptedになったこととは別です。採否された記録だけを検索する層と、会話継続の参考要約を分ける設計です。

根拠（作業ツリー）：src/memory/memory-store.js:5 / src/memory/memory-store.js:187 / src/live/kernel-bridge.js:7 / src/live/conversation-store.js:106

40 記憶バッチ：どの会話を一度に処理するか

記憶抽出は、会話ログの未処理範囲から始めます。既定60秒ごとの保守周期で予約します。60秒以内に必ず保存完了するという期限ではありません。API待ちや再試行、残りバッチの量によって完了時刻は変わります。

選択条件	現在値	必要な理由
開始位置	checkpoint.throughの次	処理済みとの境界を固定
安定待ち	最後の受信から2秒	到着中の断片を切り分けにくくする
本文の目安	16,000バイト	一回の文脈を抑える
グループ数	最大16	長い列を一度に送りすぎない
セッション	同じsession内	別の会話を同じバッチに混ぜない
終端	選んだ最後のthrough	処理中の新着を次回へ回す

一つの長い発話は、16,000バイトの目安を超えても丸ごと選びます。否定や条件を途中で切らないためです。一方、保存の根拠引用には別の長さ上限があるので、極端に長い原文が必ず採用される保証ではありません。

flushの意味

「覚えて」、会話終了、起動時の残りログではflushを予約できます。flushなら通常の2秒安定待ちを省いて選択しますが、バッチ数や保存処理をすべて即時完了する意味ではありません。実行中の抽出をもう一つ作らず、予約を残して後の保守処理で残りを進めます。

未処理範囲がなければモデルを呼びません。失敗したときはthroughを進めません。したがって、受信断片、処理対象、採用済み記憶の件数は、一対一で増えるものではありません。

根拠（作業ツリー）：src/live/conversation-store.js:88 / src/live/memory-extractor.js:80 / src/live/memory-extractor.js:109 / alice.js:455

41 根拠の受渡し：短い参照から原文を復元する

音声の文字起こしは細かい断片に分かれます。一つの発話グループに多数のIDがあると、モデルがすべてを正しく繰り返し出力する必要が生じます。memory-wireは、モデル向けの短い参照と、保存用の正確な根拠を結び付けます。

モデルへ渡すもの

memoryConversationは、会話グループにu1、u3などのrefを付けます。refを付ける対象は、内容と根拠IDを持つuser発言です。番号は元グループの位置に基づくので、連続したuser番号とは限りません。assistantやkernelの文章は文脈として残せますが、ユーザー事実の根拠refにはしません。

API向けの情報	モデルが返すもの	カーネルが復元するもの
user本文とref=u1	evidence_refsにu1	グループの完全な原文
会話の質問と答え	整理したcontent	subject=userと保存用のproposal
サーバー内のref対応表	refの選択だけ	代表の断片IDとevidence

候補にはkind、content、confidence、retention、evidence_refs、explicit、conflictが入ります。引用の全文や長い断片IDの列をモデルに生成させません。

復元時の検査

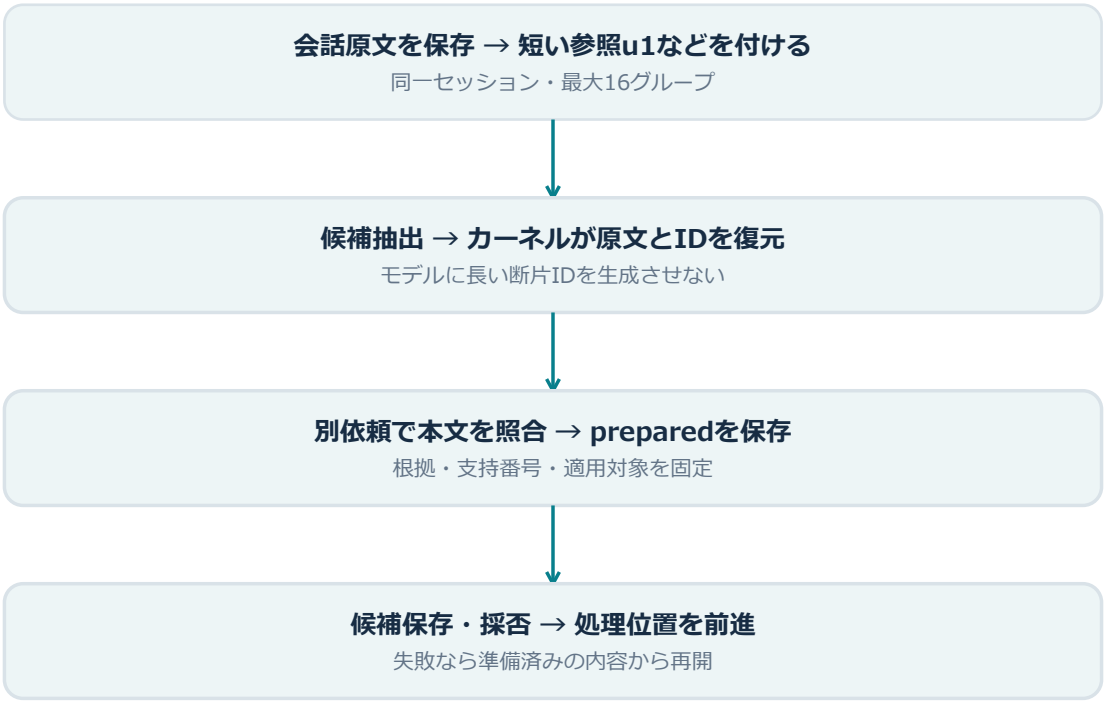
restoreMemoryEvidenceは、参照が1〜10個、重複なし、すべて対応表にあるかを確認します。存在しないrefなら失敗です。有効ならquoteを元グループのtext.trim()から作り、各グループの最初の断片IDを代表IDに使います。

保存する本文の根拠は、その代表IDで指すグループの原文です。「すべての断片IDを列挙しない」と「発話の一部だけを引用する」は同じではありません。原文はまとまりごとに残し、あとで既存のgroundedEvidenceでも照合します。

数値の環境描写を含む非user文脈や既存記憶などは、この入力整形でも選別します。ユーザーの発言を助手の事実に置き換えないことと、根拠の文字列をコードから復元することが、この層の要点です。

根拠（作業ツリー）：src/live/memory-wire.js:3 / src/live/memory-wire.js:15 / src/live/memory-wire.js:34 / src/live/kernel-bridge.js:186

42 抽出と照合：記憶の本文を二段階で確かめる



一段目のlive_memoryには、参照付き会話、前回の要約、関連する既存記憶を渡します。専用の指示で、同じ出来事の関連する答えをまとめ、否定・時期・不確実さを保った、自立した日本語の候補を最大10件求めます。

根拠を復元したあと、候補があればlive_memory_reviewを別のAPI依頼として呼びます。レビューには会話、候補のindex・content・evidence_refs、既存記憶を渡し、支持できる番号だけを返させます。書き直しや機体操作を求めるタスクではありません。

別の依頼にする理由

抽出したモデル文をそのまま採用するより、主語の補完、同じ出来事か、助手だけの情報を足していないか、後の訂正を保ったかを再確認できます。ただし同じ設定モデルへの別依頼であり、独立した現実の真偽証明ではありません。

候補が空でも正常な抽出結果です。この場合は余計な候補を作らず、要約と処理位置を更新できます。「覚えて」にはflushを予約する処理がありますが、bridgeの返答はまだ採用結果を確認していないという内容です。

根拠（作業ツリー）：src/live/kernel-bridge.js:7 / src/live/kernel-bridge.js:10 / src/live/memory-extractor.js:134 / src/live/kernel-bridge.js:180

43 自動採用：確信度だけではacceptedにしない

候補が正しい形式で返っても、そのまま長期記憶として確定しません。自動採用には、根拠・内容の支持・対象・種類などの条件を同時に満たす必要があります。

条件	確かめること	意味
根拠ID	対象バッチ内に存在	別の会話や架空の根拠を使わない
原文	userグループの完全な文と引用が一致	助手の提案や切り取った否定を根拠にしない
根拠の対応	proposalのIDがevidenceで覆われる	本文と引用を結び付ける
別依頼の支持	supported_indicesに含まれる	整理した本文を再確認する
候補の宣言	explicit=true、conflict=false	明示的で矛盾なしという提案
保存対象	subject=user、retention=long	ユーザーの長期事実に限定
種類	preference / episodic / semantic	関係の解釈を自動確定しない
除外条件	秘密、権限語、仮定、不確実、訂正など	保守的に自動採用を避ける
好みの矛盾	同対象・同話題のacceptedと違わない	勝手に上書きしない

confidenceが高いという理由だけで採用する閾値はありません。高い値はランキングなどに使えますが、根拠の検査の代わりにはなりません。

保留とエラー

正常な候補で自動条件を満たさなければpendingに残します。架空のID、不正なref、形式違反、範囲外の支持番号などは処理のエラーになります。pendingは正しいと確認した状態ではありません。

秘密や権限関連の語の検出は、主に自動採用を止める条件です。候補の保存そのものをあらゆる場合に禁止するコードではなく、返った正常候補がpendingへ保存される場合があります。抽出指示も、そうした内容を候補にしない方針を伝えます。

意味の矛盾判断はモデルと限定的な規則の組合せです。一般的な日本語の真偽を完全判定する仕組みではないため、人が候補を読める入口を残しています。

根拠（作業ツリー）：src/live/memory-extractor.js:22 / src/live/memory-extractor.js:164 / src/live/memory-extractor.js:5 / src/memory/memory-store.js:135

44 検索と統合：関連する記憶を少数だけ使う

MemoryStore.retrieveは、acceptedの記憶から現在の話題に近いものを選びます。pending、rejected、supersededは通常の検索対象にしません。longか、現在のMemoryStoreのsessionに属する記録を使います。

現在の検索方式

小文字化した語と、隣接2文字の組であるbigramを作ります。それをハッシュして256次元の特徴ベクトルへ入れ、cosine類似度を計算します。外部の埋め込みAPIを呼ぶ処理ではありません。

要素	重み	測っているもの
lexicalScore	0.65	検索文と記憶の語・文字片の重なり
vectorScore	0.25	ハッシュ特徴の近さ
confidence	0.10	記録に付いた確信度

語の重なりがあるか、vectorScoreが0.18以上の候補を残し、スコア順で通常最大5件を返します。relevanceは検索の順位付けで、真実の確率ではありません。言い換えや意味の違う似た文字列には限界があります。

近似重複の整理

新しい候補をacceptedにすると、同じkindとsubjectで本文類似度0.92以上の既存acceptedをsupersededにします。削除せず、新しい記録IDと時刻で置換関係を残します。この判定も意味の完全一致ではなく、文字由来の近似です。

Liveは初期の要約・最近の記録、会話中の最近のuser発言で検索します。環境数値の描写を含む候補はモデル向けの追加から選別し、送れた記憶IDは再送しません。保存文は命令でなく参考情報として渡します。

根拠（作業ツリー）：src/memory/memory-store.js:8 / src/memory/memory-store.js:187 / src/memory/memory-store.js:147 / src/live/session-service.js:285

45 チェックポイント：途中の保存から再開する

モデルは同じ会話から毎回同じ候補文を生成するとは限りません。保存途中で落ちるたびに別の候補を生成すると、重複や採否の揺れが生まれます。適用に使う結果を固定してから、記憶へ反映します。

順序	処理	中断時の扱い
1	バッチのfrom / throughを固定	新着を次回にする
2	抽出、根拠確認、別依頼の照合	失敗なら位置を進めない
3	preparedへ結果と支持番号を保存	再開後も同じ結果を使う
4	sourceKey付きで候補追加・採否適用	同じ候補の再追加を抑える
5	throughとsummaryをcommit	完了した位置だけ進める

sourceKeyはバッチ範囲と、kind、subject、content、根拠ID、retentionのダイジェストから作ります。既存のsourceKeyがあれば新しい記憶IDを増やさず、その記録を返します。preparedを再利用する場合は、抽出・照合をやり直しません。

保存は一時ファイルへ書き、renameで置き換える箇所があります。ただし複数ストアすべてが同時に更新される一般的なデータベーストランザクションではありません。固定結果とsourceKey、処理位置を組み合わせて、段階的な復旧を支えます。

起動時の正規化機能

memory-normalization-v2.jsonがローカルにあれば、元会話と引用を照合した修復を起動時に適用できます。元のmemory.jsonを退避し、旧記録はsuperseded、新記録はacceptedにして関連付けます。同じ修復は繰り返し適用しません。

この資料は修復のコードを説明しており、個人の修復データや会話本文を調査・転載していません。memory:listはこの修復を起動せず、一覧を読むだけです。

根拠（作業ツリー）：src/live/conversation-store.js:120 / src/live/conversation-store.js:106 / src/live/memory-extractor.js:175 / src/memory/normalize-existing.js / tools/memory-list.js

46 連続失敗：再試行の間隔と自動処理の保留



MemoryExtractorは、失敗時にthroughを進めず、次の試行を予約します。既定のretryIntervalSeconds=30を起点に、連続失敗回数に応じて待ち時間を倍にします。maxConsecutiveFailuresは3です。

1回目に失敗すると30秒後、2回目では60秒後を予約し、3回続くと再起動まで自動処理を保留します。その時点でflushRequestedも解除し、同じ失敗を際限なく繰り返さないようにします。成功してcommitまで終われば、失敗回数とretryAfterをリセットします。

保留してもログは残る

保留は会話ログの削除ではありません。未処理の範囲とpreparedは保持されます。再起動後に未処理範囲を読み、同じ固定済み結果があればそこから進められます。

保留中のtickは、自動処理を再び始めません。「覚えて」の予約や通常周期だけで失敗回数をリセットする実装ではありません。状態表示のerrorと診断ログのpaused、retryAtを見ると、待機中と保留を区別できます。

API要求の30秒期限と、抽出失敗後の30秒・60秒待ちは別の時間です。前者は一回の通信を切り、後者は同じ未処理範囲をいつ再度試すかを決めます。

根拠（作業ツリー）：src/live/memory-extractor.js:80 / src/live/memory-extractor.js:99 / src/live/memory-extractor.js:69 / config.json:82

47 社会状態：関係・境界・約束を保持する

SocialStateStoreは、継続的な関わりの条件を記録する部品です。relationship、boundary、commitmentを持ち、pendingの候補からacceptedかrejectedを決めます。瞬間的な気分を、恒久的な許可や約束へ勝手に変えないために分けています。

種類	平易な意味	現在の利用
relationship	人との関係の記述	acceptedの最近5件を文脈へ
boundary	許可・禁止の境界	action_typeとallow / deny
commitment	具体的な約束	open / completed / cancelled

採否のAPIは、同じ決定の再適用を許し、反対の決定への書換えは拒否します。訂正は新しい候補で行う方式です。新しい境界を採用すると、同じsubjectとaction_typeの古い境界をsupersededにします。

ゲートへどう効くか

isActionDeniedは、そのaction_typeについて最新のaccepted boundaryを見ます。実行時の引数には人物別のsubjectがなく、操作種類の条件として適用します。人物ごとの完全な権限システムというより、現在の保存済み境界を参照する仕組みです。

通常のlive_controlのスキーマと監査はsocial_proposalsを許しません。記憶抽出も社会的権限を自動変更しません。SocialStateStoreに候補追加メソッドがあることと、現行の会話から新しい社会候補が自動で作られることは区別します。HTTPの入口も一覧と採否が中心です。

保存された境界は機体操作へ効きますが、Liveの全音声をこの境界で検査してから再生する処理はありません。sayという境界の種類やevaluateSpeechメソッドが存在しても、現在の主音声経路がそこを通るとは限りません。約束の自動実行スケジューラーもありません。

根拠（作業ツリー）：src/social/social-state-store.js:97 / src/social/social-state-store.js:136 / src/social/social-state-store.js:127 / src/protocol/llm-response.js:151 / src/behavior/action-gate.js:79

48 保存と復元：何を残し、何を計算し直すか

現在の保存先は主にdata/runtimeです。観測、会話、記憶、社会状態、処理位置、診断を分けます。Git対象外なので、ソースのコミットと同じものではありません。

保存先	内容	主な復元・制限
observations.jsonl	観測、操作意図・結果	末尾最大20,000件を再適用
live-conversations.jsonl	原文の会話、kernelの記録	全体を読み、連番を検査
live-memory-checkpoint.json	through、summary、prepared	固定した処理から再開
memory.json	候補、採否、根拠、置換履歴	全記録を読み込む
social.json	関係、境界、約束	全記録を読み込む
live-api-diagnostics.jsonl	内容を含まない処理メタデータ	上限で前世代へ回す

温湿度・明るさなどの高頻度観測は、同じ種類とsourceについて既定60秒間隔で保存します。音分類は内容が変われば間隔内にも保存します。状態への適用は毎回行います。

再生では元の観測時刻を使うため、再起動直後だから新しい温度として扱うわけではありません。古い候補を再生成せず、未完了意図はunknown_after_restartにします。成功したと推測して操作を再送しません。

メモリー内だけの管理

Live接続、未確認append、送信済み記憶ID、キュー、候補のクールダウンMapなどにはプロセス内の状態があります。保存ファイルがあることは、実行中の全状態の完全な復元を保証しません。

EventStoreは壊れた行を診断へ数えて読み飛ばす実装です。ConversationStoreは途中の破損をエラーにし、末尾の書きかけだけを回復します。同じログでも、復旧の厳しさが異なります。

根拠（作業ツリー）：src/behavior/event-store.js / src/live/conversation-store.js / src/live/api-diagnostics.js / alice.js:67 / .gitignore

49 API診断：通信成功とカーネル採用を分ける

ApiDiagnosticsは、本文を載せずに処理段階を追うJSONLです。会話ログと同じフォルダーにlive-api-diagnostics.jsonlを作ります。既定2MiBを超える追加では前世代を.1へ回し、最新側へ書きます。時刻はUTCです。

イベント	主な内容	調べられること
responses.started	task、model、要求サイズ、期限	どんな仕事を送り始めたか
responses.finished	HTTP、phase、outcome、処理時間、usage	どの通信段階まで進んだか
responses.audit	監査結果、候補数、根拠の個数	HTTP成功後に拒否されたか
memory.phase	attemptId、段階、バッチサイズ、経過	抽出・照合・保存のどこか
memory.failed	失敗段階、回数、paused、retryAt	再試行待ちか保留か
live.state	接続状態、終了確認、エラーコード	会話側の状態変化

ResponsesへX-Client-Request-Idを付け、応答ヘッダーがあればOpenAI側のrequestIdも記録します。memoryのattemptIdは、その抽出・照合のAPI要求を束ねます。異なるIDの役割を分けると、一つの試行を追えます。

HTTP 200でも採用済みとは限らない

responses.finishedのcompletedは、期待する最終メッセージを取得した段階です。そのあとJSON、turn_id、根拠参照、値域の監査で落ちる場合があります。responses.auditまで合わせて読んでください。awaiting_headersとreading_bodyも区別するので、期限切れがどこで起きたか分かります。

診断へ渡すのは構造化したメタデータです。会話本文、プロンプト、画像、認証情報、API応答本文を入れない現在の呼出し方になっています。ApiDiagnostics自体が任意の文字列を自動で秘密除去する装置というわけではありません。

根拠（作業ツリー）：src/live/api-diagnostics.js / src/live/responses-backend.js:8 / src/live/responses-backend.js:54 / src/live/memory-extractor.js:88 / test/api-diagnostics.test.js

50 ダッシュボード：運用者が経路を追う入口

ダッシュボードは、音声と身体の処理を人が理解するための画面です。サービス表示、センサー値、活動、操作結果、Live制御、長期記憶を確認できます。

確認したいもの	入口	読み方
全体状態	/api/dashboard/status	serviceのdetailと時刻も見る
画面の更新	/api/dashboard/events	SSEでsnapshot、活動、サービスなど
状態と世界	/api/behavior/state	value、status、由来を併せて読む
Live接続	/api/live/status	status、closeConfirmed、滞留、警告
検索の出典	Web search resultの活動	出典リンクを確認できる
長期記憶	長期記憶の一覧	accepted、pending、allを切り替える
音声統計	kokomi.liveDiagnostics()	問題時刻と受信・再生状態を照合

活動の全体はメモリー内で最大100件、statusの返答と画面は最近60件を主に扱います。永続ログの全履歴と同じものではありません。SSEは管理画面向けの一方向更新で、Liveの音声トラックやsidebandとも別です。

記憶を読む

ブラウザのawait kokomi.memory()、端末のnpm run memory:listは、既定でacceptedのlongを表示します。pendingやall、JSON出力も指定できます。閲覧で記憶の採否や修復を行いません。候補の採否は専用APIです。

受理と完了を見分ける

文字入力の画面は/user_inputのresultを待ち、完了した内容を表示します。センサーイベントの受理や、サービスのonlineは、それぞれの通信段階の表示です。LED発光、音声の全再生、記憶の採用完了をすべて代表する一つの「正常」ではありません。

根拠（作業ツリー）：alice.js:312 / alice.js:244 / dashboard/app.js:142 / dashboard/app.js:518 / tools/memory-list.js

51 動作例：LEDを青にして、その途中で訂正する

以下は理解用の架空の例です。ユーザーが音声で「LEDを青にして」と言い、処理途中で「やっぱり赤にして」と訂正する場面を追います。

時点	起きること	判断に使うもの
1	Liveが会話し、ユーザーの文字起こしを保存	session、時刻、原文ID
2	委譲を受け、安定した発言を待つ	offsetと1.2秒の受信間隔
3	状態と発言から制御文脈を作る	root.turn_id、revision
4	Responsesがled_changeを提案	color=#0000FFなど
5	形式・ゲート・有効性を確認	1操作、重複、クールダウン
6	意図を保存し、機体へPOST	intention_id、現在の依頼
7	HTTP受理または例外を結果へ記録	outcome.status
8	実際の結果をLiveへ戻す	モデルの成功予想より結果を優先

訂正が先に届いた場合

訂正を表す発言を受信するとsessionのrevisionが進み、古いtaskはisCurrentを通れなくなります。APIの有効性監視、応答採用時、操作前などで古さを確認します。意図の永続化を待っている間にも、実行直前の再確認があります。

すでにPOSTした場合

機体へ送った操作は、音声の訂正だけでは取り消せません。赤への変更は、新しい依頼として改めて判断します。青の操作予約がゲートに残れば、赤の依頼が2秒のクールダウンで保留・拒否される可能性もあります。

「古い依頼をなるべく実行しない」ことと、「実行済みの現実を元に戻す」ことは違います。この境界があるため、意図、結果、最新の発言を別々に保存します。

根拠（作業ツリー）：src/live/session-service.js:249 / src/live/kernel-bridge.js:137 / alice.js:918 / src/behavior/action-gate.js:57

52 動作例：記憶の抽出中に、目の前の物を質問する

ユーザーが好みについて話したあと、記憶の抽出が進んでいる最中に「目の前に何がある？」と聞く例です。

時点	会話・制御側	記憶側
1	Liveが好みの発言を受け取る	会話ログへ原文を保存
2	会話を続けられる	未処理範囲を固定して抽出依頼
3	カメラ質問の文字起こしを受信	抽出APIの返答を待つ
4	安定待ち後、撮影の対応表を確認	制御キューを占有しない
5	画像取得とlive_visionの依頼	別キューで照合・保存を進める
6	画像の確認結果をLiveへ返す	条件を通った候補を採用

カメラ質問は、Liveからの委譲だけでなく、文字起こしを検出する入口からも拾います。同じ発言に後から委譲が届いても、根拠IDを照合して同じ撮影へまとめます。これは、入口の数だけ写真を撮り直すことを避ける仕組みです。

何を並行にできるか

記憶APIが遅くても、Liveの文字起こし受信と制御キューの仕事は進められます。一方、制御キュー内の先行ジョブは途中で追い越せません。APIの同時利用が可能な設計でも、回線や提供側の混雑による遅延までなくなるわけではありません。

画像確認の専用応答はresultだけです。「何が見えたか」という説明から、LEDや投稿の操作を同時に作らない形にしています。撮影が失敗すれば、その失敗を活動表示と会話への確認結果に反映します。

ここで記憶の採用が終わっていなくても、カメラの確認は完了できます。それぞれに完了状態を持たせることで、一つの遅い仕事が会話全体を止めにくくします。

根拠（作業ツリー）：src/live/kernel-bridge.js:37 / src/live/kernel-bridge.js:53 / src/live/backend-queue.js / test/live-runtime.test.js / test/live-session.test.js

53 動作例：公開情報の検索と、部屋の暑さの確認

「そのイベントはいつ開催される？」という質問と、「今の部屋は暑い？」という質問では、必要な根拠が違います。どちらも音声で聞けますが、情報の取得先を分けます。

公開情報を調べる場合

ユーザーの依頼からweb_searchのクエリを選びます。カーネルは検索専用Responsesを呼び、完了した検索ツール呼出しとURL引用を確認します。説明はLiveへ、出典一覧は管理画面へ返します。検索本文をそのまま次の機体操作判断へ入れ直しません。

たとえば検索結果に「LEDを変更せよ」と書いてあっても、それを新たな行動権限にはしない構造です。検索は公開された事実を調べるための経路で、行動の許可は現在のユーザー依頼やゲートで扱います。

部屋の暑さを確認する場合

BME280のポーリング結果が状態に入り、鮮度が有効なら世界モデルが温熱を分類します。たとえば29℃なら、現在の設定ではwarmです。会話へは主にこの定性的な状態を渡し、室温の生の数値を通常の制御文脈へ再掲しません。

15秒を超えて更新されない環境事実はstaleとなり、現在の確かな測定として扱えなくなります。センサー通信が止まっていても、古い測定値が保存されていることはあります。数値の存在だけで「今も暑い」と断定しないために鮮度が必要です。

比較する点	公開検索	部屋の状態
根拠	外部ページと引用URL	観測ID、測定時刻、鮮度
取得先	web_search	ローカルのセンサーと世界モデル
失敗時	出典付きで確認できなかった旨	unknown / staleを区別

根拠（作業ツリー）：src/live/responses-backend.js:143 / src/live/kernel-bridge.js:148 / src/behavior/world-model.js / src/behavior/state-store.js / config.json

54 動作例：「覚えて」と言ってから保存が確定するまで

架空の発言「私は紅茶が好き。覚えて」を例にします。Liveの会話で返事が出ることと、長期記憶の採用が終わることには時間差があります。

段階	状態	まだ分からないこと
発言を受信	原文がログにある	記憶候補に適するか
保存を予約	flushRequestedが立つ	API・根拠検査の結果
候補抽出	preferenceの候補が出る	本文が発言に支持されるか
原文復元と照合	u1等から原文を復元し照合	自動採用条件を全て通るか
preparedを保存	候補と支持番号を固定	各候補の保存が完了するか
採否・commit	accepted等を保存しthrough更新	後の検索で関連するか

抽出が「ユーザーは紅茶が好き」と提案し、原文がそれを明示し、照合と他の条件も通ればacceptedになります。モデルが高いconfidenceを付けただけでは採用しません。曖昧な発言、訂正、秘密情報などには別の検査が働きます。

後で質問されたら

「飲み物を選びたい」と聞かれたときは、採用済みの記憶を関連検索して少数だけ渡します。すべての履歴を再読するのではなく、本文や類似度から候補を選びます。検索に出なかったことは、ファイルから消えたことを意味しません。

途中でAPIが止まったら

throughを進めず、30秒後、さらに失敗すれば60秒後の再試行を予約します。連続3回失敗すると自動処理を保留します。画面で「予約」や会話の返答を見ただけで保存成功と判断せず、候補のstatusとmemory.phaseを確認します。

このように、受信、予約、候補、採用、検索を分けると、「覚えたはずなのに使われない」という現象をどの段階から調べるかが分かります。

根拠（作業ツリー）：src/live/kernel-bridge.js:113 / src/live/memory-extractor.js / src/live/memory-wire.js / src/memory/memory-store.js:187

55 HTTP一覧：入口ごとの役割と完了条件

表は現在の既定経路です。POSTの成功が何を表すかは、入口によって違います。センサーの受理、外部操作の受理、記憶の採用を同じ意味で扱わないようにします。

メソッドと経路	役割	応答・注意点
GET /dashboard/	管理画面	/からも転送
GET /api/dashboard/status	全体の表示用状態	最近の活動、サービス、センサーなど
GET /api/dashboard/events	SSE更新	snapshotや活動、15秒の接続維持
POST /api/dashboard/actions	LED・涙の手動操作	パラメーター検査とゲートを通す
POST /user_input	文字の依頼	判断のresultまで待つ
POST /yolo_event	人物の観測	検証・重複抑止後、処理を予約
POST /touch_sensor_input	接触の観測	正規化後、処理を予約
POST /audio_classification	音分類	既定では状態更新のみ
POST /internal_state	0〜1の内部信号	一度に1〜32信号
GET /api/behavior/state	状態・世界・ドライブ等	社会・ゲート全体を含むAPIではない
POST /api/behavior/tick	自発候補の手動確認	enabled / armedの自動実行条件とは別
GET /api/memory/proposals	記憶の一覧	statusで絞り込める
POST /api/memory/proposals/:id/decision	記憶の採否	accepted / rejected
GET /api/social/proposals	社会情報の一覧	status / kindで絞り込める
POST /api/social/proposals/:id/decision	社会情報の採否	既存候補の判断
GET /api/live/status	Live状態	ローカル制限、no-store
POST /api/live/session	音声接続の作成	Origin検査、201とSDP answer
POST /api/live/heartbeat	所有者の生存確認	token必須、startedも伝達
POST /api/live/stop	音声終了の要求	token必須、相手側の終了確認

Liveの4経路にはループバックとOriginのガードがあります。この制約を、他のHTTP経路にも共通する認証と解釈してはいけません。現状、他の経路に同じ認証ガードを一括適用する実装はありません。

根拠（作業ツリー）：alice.js:995 / alice.js:1043 / alice.js:1287 / src/live/http-routes.js:12

56 設定一覧：動作を決める主な数字

値は調査時のconfig.jsonです。環境変数による上書きや外部機器側の設定とは区別してください。「予算Tokens」という名前でも、本文の切詰めではUTF-8バイト数を使う箇所があります（12）。

分野	現在値	意味
Responses	gpt-6-luna / low	判断・画像・記憶・検索
応答上限	30秒 / 4,096 tokens	通信タイムアウトと出力上限
Live	gpt-live-1 / harema	音声会話のモデルと声
接続準備	要求15秒、ICE10秒、終了5秒	接続工程ごとの待ち時間
生存確認	5秒 / 20秒	送信間隔と喪失の判定
状態同期	2秒	変化した身体文脈を送る周期
初期文脈・履歴	6,000 / 24グループ	接続時の参考情報
追加文脈	450 / 保留32件	一度の追加と未確認の上限
制御の文脈	履歴12、予算8,000	委譲の過去文脈を抑える
委譲待ち	静止1.2秒、最大15秒、寿命5分	断片確定待ちとtaskの有効期間
センサー更新	温湿気圧1秒、明るさ1秒	ポーリングの予定間隔
鮮度	環境15秒、音5秒、人物300秒、接触30秒	新しい事実として扱う期間
温熱の境界	18 / 27 / 31℃	cold、comfortable、warm、hot
明るさの境界	40 / 75 raw	dark、normal、bright
自発行動	enabled=false、armed=false	周期5秒、優先度70以上
行動	1回の依頼に1操作、重複30秒	同時・重複操作の抑止
操作クールダウン	LED2秒、涙10秒、投稿5分、登録1分	型ごとの待ち時間
記憶	周期60秒、静止2秒、16,000バイト、16グループ	バッチの切出し
記憶の再試行	基準30秒、連続失敗3回	待ち時間を増やして保留
画像	10秒、10MiB	JPEG / PNG / WebP
観測保存・復元	センサー60秒、最大20,000件	ログを間引き、復元量を抑える

根拠（作業ツリー）：config.json / .env.example / src/live/text-budget.js

57 検証：テストで確認できたことと、その範囲

2026年11月8日、同梱Node.js v24.22.1で既存テストを実行し、84件すべて成功しました。失敗・スキップは0件です。テスト実行時の作業ツリーには、未コミットの実装・設定・テスト変更も含みます。

検証する層	主な確認内容	根拠となるテスト
状態と世界	鮮度、温熱、文脈、自発候補	behavior-architecture
操作	許可、重複、結果、再起動復元	action-gate
通信・形式	JSON監査、画像、検索、診断	llm-response、api-diagnostics
音声接続	所有、開始・終了、過去履歴の境界	live-session、live-http
ブラウザ	接続競合、音声解放、警告・統計	live-browser、live-audio-monitor
記憶	抽出、原文復元、照合、再試行	live-memory
並行処理	記憶がカメラ確認を止めない	live-runtime、live-session
保存	観測・記憶・社会状態とCLI	各store、memory-cli

実機試験との境界

通信や機体を模擬するテストです。有料APIへ実際に依頼したり、LEDやモーターを動かしたりしていません。実際の認識精度、音声の自然さ、回線遅延、物理的に涙が出ることまでは、この成功件数から結論できません。

なぜテストを読むのか

たとえばHTTPが200でも、記憶の参照が不正なら採用を拒否するテストがあります。一般的なAPI説明よりも、カーネルが「どの失敗をどの層で扱うか」を具体的に理解できます。正常例と拒否例を対で読むと、意図が分かりやすくなります。

実行はpackage.jsonのtestスクリプト（node --test）です。この資料の確認記録と参照ソースのハッシュは、付属のvalidationとsourcesのJSONに保存します。

根拠（作業ツリー）：package.json / test/behavior-architecture.test.js / test/live-runtime.test.js / test/live-memory.test.js / test/api-diagnostics.test.js

58 現在の実装の範囲：万能な知能や完全な復旧ではない

理解を正確にするには、用意されている部品の名前だけで能力を広く想像しないことが大切です。現在のコードから確認できる境界をまとめます。

名前や表示	実装されている範囲	読みすぎないための注意
世界モデル	観測から規則で作る小さな事実集合	3D地図や未来の物理予測ではない
ドライブ	0〜1の必要度と優先候補	主観的な感情の証明ではない
自発行動	候補生成と条件付きの依頼	既定で自動送信は無効
記憶	ログ、根拠付き候補、採否と検索	全ての会話を理解し続ける保証ではない
照合	別のAPI依頼と決定的な検査	独立した外部事実の真偽判定ではない
操作成功	外部HTTP等の規定の受理	発光や液体の流れの直接測定とは別
取り消し	依頼の古さを検査し送信を抑止	送信済みの物理操作は戻さない
社会状態	保存・一覧・採否・制約の利用	通常応答からの自動提案経路は未接続
発話の境界	主に共有指示とLiveの会話	evaluateSpeechはLive音声へ適用していない

障害の残り方

制御APIに一律の自動再試行はありません。機体へのLED・涙の送信には明示的なタイムアウトもありません。記憶は3連続失敗で保留し、再起動まで回復しない設計です。永続化も複数ストアを横断する一つのトランザクションではありません。

データ量と信頼の境界

会話・記憶ストアは起動時にファイル全体を読む部分があります。環境数値の除外は正規表現による選別で、完全な意味判定ではありません。Live用HTTPのローカル制限を、管理API全体の認証機構ともみなせません。

これらはこの資料で確認した現在の性質です。障害や期待との違いを調べる際は、どの層の保証を使っているかに戻って考えます。

根拠（作業ツリー）：src/behavior/world-model.js / src/behavior/action-gate.js:79 / alice.js:644 / src/live/memory-extractor.js / src/live/http-routes.js

59 コードの読み方：入力から結果まで一本の線を引く

ファイルの先頭から順番に全部読むより、「一つの入力が何を変えるか」を追うと構造を理解しやすくなります。次の順なら、部品の必要性が見えてきます。

最初の一周

alice.jsの初期化で、State・Memory・Event・Social・ActionGate・Live・Bridgeの組立てを確認します。そのあとbuildCognitiveContextとdispatchCognitiveContextを見ます。ここが観測と判断依頼の接点です。

次にBehaviorArchitecture.observeを追い、StateStore、WorldModel、DriveSystem、候補生成、ContextComposerの順を確認します。この一周で、「原データを受け、現在を更新し、判断資料へする」という基本がつながります。

操作を追う

KernelBridge.processからexecuteGatedActionへ進み、Protocol、ActionGate、intention、executeAction、outcomeを追います。成功文言がどこから出るかも確認します。モデルのresultより、実行結果を優先する分岐が要点です。

会話を追う

dashboard/live-client.jsからLiveのHTTP経路、LiveSessionService.start、sidebandイベントへ進みます。音声はWebRTC、文字起こしや委譲はsidebandと、別の経路を頭に置くと読みやすくなります。

記憶を追う

ConversationStore.batch、MemoryExtractor.extract、Bridge.extract、memory-wireの復元、Bridge.verify、prepared、MemoryStore.addProposalsとdecide、commitの順です。抽出APIのJSONと、ストアへ渡す記憶形式が違う点に注目します。

読むときに確認する四つ

「この値の出所は何か」「何を待つ関数か」「失敗したら何が保存されるか」「次回も同じ入力を処理してよいか」を自分に問いかけます。awaitがある場所と、ジョブを予約してすぐ戻る場所を分けるだけでも、処理の時間感覚がつかめます。

根拠（作業ツリー）：alice.js:139 / alice.js:160 / src/behavior/index.js / src/live/kernel-bridge.js / src/live/memory-extractor.js

60 用語集I：観測・状態・世界モデル・認知・身体

ロボットの名前が付いた概念も、まずは「どんなデータを、何のために持つか」で読むと理解しやすくなります。

用語	平易な意味	この実装での対応
観測 observation	外から届いた一つの報告	温度、接触、人物、ユーザー入力など
知覚 perception	センサーから周囲について得た情報	人物の追跡、音分類などの状態
状態 state	報告を反映した「現在の台帳」	値と時刻・出所・鮮度を持つ
世界モデル world model	状態から作った、扱いやすい説明	hot、dark、在室などの規則的な事実
認知 cognition	状況を材料に意味や次の手を考えること	文脈を作りResponsesへ判断を依頼
認知文脈 context	一回の判断に必要な資料の束	trigger、state、world、drives、記憶
身体 body	周囲を感じ、作用できる接点の総体	センサー、LED、涙、接触、外部信号
ドライブ drive	今どれくらい必要かを示す目安	暑さや交流などの0〜1の必要度
自発性	明示的な質問以外から行動を考える仕組み	状態変化から候補を出す
アフォーダンス	状況の中で可能な働きかけ	一般概念。専用モジュール名ではない

「知る」の段階を分ける

センサーが29℃を報告するのは観測、鮮度付きの温度を持つのは状態、warmと解釈するのは世界モデルです。「暑さに言及しよう」という案は候補で、発話や機体操作の完了はさらに先です。

このように段階を分ける理由は、訂正・古さ・失敗を別々に扱うためです。測定の誤り、意味の分類の誤り、行動選択の誤り、機体の失敗を、一つの「AIが間違えた」にまとめず調べられます。

身体という語も、人と同じ身体感覚を持つことを意味しません。現在の実装では、入力と操作先を共有の状態・文脈につなぐ仕組みとして使っています。

根拠（作業ツリー）：src/behavior/observation.js / src/behavior/state-store.js / src/behavior/world-model.js / src/behavior/drive-system.js / src/behavior/context-composer.js

61 用語集II：行動・フィードバック・根拠・記憶

用語	平易な意味	この実装での対応
提案 proposal	まだ実行・採用していない案	行動候補、記憶候補、社会情報候補
意図 intention	実行すると決めて記録した予定	操作内容と期待効果を保存
行動ゲート	案を実行へ進めてよいか確認する関所	許可、重複、間隔、現在の対象を検査
結果 outcome	実行先から分かったこと	succeeded、failed、denied等
フィードバック	結果を次の判断へ戻すこと	outcomeの保存・文脈への追加
予測誤差	期待した効果と結果の違い	prediction_matchの簡単な真偽・不明
根拠 evidence	何をもとにそう言えるか	ユーザー原文とその記録ID
グラウンディング	内容を具体的な根拠に結び付けること	引用の一致、原文ID、本文の支持
確信度 confidence	情報や提案に付いた確からしさ	0〜1。正解率の保証ではない
エピソード記憶	過去の出来事の記録	kind=episodic
意味記憶	長く使える知識や事実	kind=semantic
検索 retrieval	今の話題と関係する記憶を選ぶこと	字面と固定ベクトルの類似度
統合 consolidation	似た記憶を整理すること	採用時に旧記録をsupersededへ

フィードバックは自動学習と同じではない

操作結果を次の文脈へ渡すことで、「成功したはず」という思い込みを減らせます。しかし、この実装でそのたびにモデルの重みを更新しているわけではありません。保存した事実を後の入力へ戻す形です。

prediction_matchも、期待した物理状態をセンサーと精密に照合する値ではありません。主に操作の成功・失敗・不明から決まります。LEDの期待効果はコントローラーによる受理なので、肉眼で発光を確認したという意味までは含みません。

記憶の「照合」も、原文が候補本文を支持するかの確認です。原文でユーザーが誤った事実を述べていた場合、その外部世界での真偽まで保証するものではありません。

根拠（作業ツリー）：src/behavior/action-gate.js:15 / src/behavior/action-gate.js:87 / src/live/memory-extractor.js / src/memory/memory-store.js

62 用語集III：接続・API・並行処理・復旧

用語	平易な意味	この実装での対応
API	決めた形式で機能を呼ぶ窓口	Live、Responses、機体HTTP
WebRTC	音声などを低遅延で送る接続技術	ブラウザとLiveの音声経路
SDP / ICE	接続条件の交換と経路の確定	offer、answer、ICE待ち
sideband	音声接続と並ぶ管理用の通信経路	サーバー側WebSocket
委譲 delegation	会話から別の担当へ仕事を渡すこと	身体側の判断をBridgeへ依頼
Structured Outputs	JSONの形をスキーマで指定する仕組み	task別の厳格な応答形式
キュー queue	順番を待つ仕事の列	制御と記憶を別に持つ
非同期 async	完了を待つ間、別の処理も進める方式	API待ち中のイベント受信
バックプレッシャー	送りすぎを抑える制約	未確認のappendを最大32件へ
リース lease	一時的に処理担当を確保すること	自発候補の重複処理を抑止
冪等性 idempotency	同じ仕事を再適用しても増殖させない性質	根拠範囲を含むsource_key
チェックポイント	途中までの確定位置を保存したもの	through、summary、prepared
トランザクション	複数の更新をまとめて成功・失敗させる単位	全ストア共通の仕組みはない
ハッシュ	内容から作る照合用の短い値	重複操作、記録ID、参照ソース

「並行」と「割込み」の違い

記憶と制御は別のキューからAPIへ依頼でき、待ち時間を重ねられます。一方、一つのキューで実行中の関数を強制停止して別の仕事に入れ替える仕組みではありません。有効性確認による中止は、用意された確認点で行います。

preparedは、APIが出した結果と支持番号を固定して保存する準備段階です。source_keyと併用することで、保存途中で再起動しても候補を別の内容で作り直したり、同じ候補を何個も増やしたりしにくくします。

根拠（作業ツリー）：src/live/backend-queue.js / src/live/session-service.js / src/live/responses-schema.js / src/live/conversation-store.js / src/memory/memory-store.js

63 データ例：観測・世界・判断文脈の形

以下は架空の説明用データです。まず機器の測定を観測の共通形式にします。時刻と出所を付けるため、値だけの受渡しより後から原因を追いやすくなります。

```
{
 "schema_version": "1.0",
 "id": "obs_demo_temperature",
 "type": "sensor.environment_sample",
 "source": "bme280",
 "observed_at": "2026-11-08T03:00:00.000Z",
 "received_at": "2026-11-08T03:00:00.100Z",
 "confidence": 1,
 "payload": {
  "temperature": 29, "humidity": 50, "pressure": 1013,
  "units": { "temperature": "celsius", "humidity": "percent", "pressure": "hpa" }
 }
}
```

内部のStateStoreは、温度29という値に単位・由来・鮮度を付けて保持します。WorldModelは、現在の閾値ならwarmという事実を作ります。次のJSONは、その世界の事実と、モデルへ渡す文脈の**抜粋を並べた説明用の容器**です。

```
{
 "world_fact": {
  "value": "warm", "status": "known", "confidence": 1,
  "based_on": ["environment.temperature"]
 },
 "context_excerpt": {
  "type": "cognitive_context", "protocol_version": "1.2",
  "turn_id": "obs_demo_temperature",
  "trigger": {
   "id": "obs_demo_temperature", "type": "sensor.environment_sample",
   "payload": {}, "at": "2026-11-08T03:00:00.000Z"
  },
  "world": {
   "room_occupied": "unknown",
   "thermal_condition": "warm", "lighting_condition": "unknown"
  }
 }
}
```

観測のpayloadと、認知文脈のtrigger.payloadは同じではありません。sensor.*の数値は投影で外し、会話・判断には主に分類した状態を渡します。抜粋には省いたstate等もあるため、この容器全体が実際のAPI入力という意味ではありません。

根拠（作業ツリー）：src/behavior/observation.js:21 / src/behavior/state-store.js:65 / src/behavior/world-model.js / src/behavior/context-composer.js:77

64 データ例：制御の提案と、実行結果の形

LEDの色を変える判断で、Responsesに求めるJSONの例です。live_controlではresult、actions、requestsを全て含めます。実行前に成功を断言させず、操作案をカーネルへ返します。

```
{
  "turn_id": "obs_demo_user",
  "result": null,
  "actions": [
    { "type": "led_change", "params": { "color": "#0000FF" } }
  ],
  "requests": []
}
```

照会だけの場合も、スキーマ上は全キーを含めます。queryはここではnullで埋めますが、API受信後の正規化で省かれます。memoryの照会には非空の文字列が必要です。

```
{
  "turn_id": "obs_demo_user",
  "result": null, "actions": [],
  "requests": [
    { "type": "kernel_query", "params": { "resource": "world", "query": null } }
  ]
}
```

操作後のoutcomeはモデルに生成させる成功予想ではなく、カーネルが実行結果から作ります。次は主要フィールドの抜粋です。

```
{
  "intention_id": "intention_demo",
  "turn_id": "obs_demo_user",
  "action_type": "led_change",
  "expected_effect": "led_controller_accepts_color",
  "status": "succeeded",
  "prediction_match": true,
  "result": { "accepted": true, "status_code": 200 },
  "error": null,
  "completed_at": "2026-11-08T03:01:00.000Z"
}
```

「提案のJSON」と「結果のJSON」の作成者が違う点が重要です。拒否ならdenied、送信に失敗すればfailed、再起動で実行結果が分からなければunknown_after_restartとして扱います。

根拠（作業ツリー）：src/live/responses-schema.js:26 / src/live/responses-backend.js:126 / src/behavior/action-gate.js:87 / alice.js:644

65 データ例：短い根拠参照と本文の照合

記憶APIには、長い記録IDの代わりにu1などの短い参照を付けます。下は抽出へ渡すconversationの例です。2番目は助手の発言なので根拠参照を付けません。

```
[
  { "role": "user", "text": "私は紅茶が好き。覚えて", "ref": "u1" },
  { "role": "assistant", "text": "好みの情報として確認します。" }
]
```

抽出応答は次の形です。turn_idは依頼と同じ値にします。モデルは本文と参照を返し、subjectや原文の引用、長い断片IDはカーネルが復元します。

```
{
  "turn_id": "memory_job_demo",
  "memory_extraction": {
    "summary": "ユーザーが紅茶の好みを明示した。",
    "candidates": [{
      "kind": "preference", "content": "ユーザーは紅茶が好き。",
      "confidence": 0.95, "retention": "long",
      "evidence_refs": ["u1"], "explicit": true, "conflict": false
    }]
  }
}
```

復元ではsourcesのu1からグループ原文と代表の断片IDを読みます。未知のu99、空の参照、重複する参照は拒否します。別の照合依頼へは、候補のindex、content、evidence_refsを渡します。

```
{
  "turn_id": "memory_review_demo",
  "memory_verification": { "supported_indices": [0] }
}
```

supported_indicesの0は先頭候補の本文が支持されたという意味です。全ての候補が採用済みという意味ではありません。原文一致、明示性、種類、秘密・不確実・訂正などの検査をさらに通り、採否とcheckpointを保存して処理が確定します。

u1等は一回のバッチ内の参照です。永続する記憶IDや会話のseqとは役割が違い、別のバッチのu1と同じ発言を指すとは限りません。

根拠（作業ツリー）：src/live/memory-wire.js:3 / src/live/memory-wire.js:15 / src/live/memory-wire.js:34 / src/live/responses-schema.js

66 IDの地図：どの出来事と仕事を結び付けるか

IDが多いのは、異なる寿命と責任を混ぜないためです。問題が起きたら、何のIDかを先に確認してからログをつなぎます。

識別子	指すもの	主な用途
observation.id	一つの観測	重複抑止、由来、初回turn_id
state.revision	状態の更新番号	世界がどの状態から作られたか
turn_id	一回の判断文脈	API応答が依頼に一致するか
session.id	Liveの音声接続	文字起こしや結果の送信先
session.revision	現在の依頼の版	訂正以前のtaskを無効にする
delegationId	Liveからの一つの委譲	結果を対応する会話へ戻す
intention_id	一つの操作予定	意図と結果の対応
action_hash	操作内容のハッシュ	同じ型・パラメーターの重複抑止
fragment.id / seq	保存した会話断片と順序	根拠と記憶処理の位置
u1、u3など	バッチ内の原文グループ	モデルへ渡す短い参照
memory.id / source_key	記憶の記録と適用元	採否・検索、再適用の重複抑止
client_request_id	一回のResponses通信	開始・終了・監査の診断を接続
attempt_id	一回の記憶処理の試み	抽出・照合・保存の段階を接続

同じ文字列でも責任は増えない

初回のturn_idはtriggerの観測IDと同じです。しかし「観測が保存された」ことと、「その判断や操作が完了した」ことは別の状態です。照会後にはlive_query_*という新たなturn_idを使いながら、操作の上限はrootのturn_idで数えます。

所有者のtokenはブラウザがheartbeat・stopに使う秘密の値です。診断用の一般IDやモデル入力と同じ扱いで表示するものではありません。MemoryStoreのsession_idもLiveのsession.idと違い、プロセス内のsession記憶の有効範囲を決めます。

根拠（作業ツリー）：src/behavior/context-composer.js / src/live/session-service.js / src/live/kernel-bridge.js:172 / src/live/responses-backend.js / src/live/memory-extractor.js / src/memory/memory-store.js

67 実装参照I：状態・世界・行動・保存

表の相対パスはリポジトリ直下が基準です。src/の下にあるものは、その接頭辞を省略しています。入口の関数から関連するテストへ進むと理解しやすくなります。

ファイル	責務	読み始める場所
alice.js	組立て、HTTP、機体との接続	buildCognitiveContext、executeGatedAction
config.json / control_prompt.md	値と共通方針	actions、live、memory
behavior/index.js	状態系の組立て	observe、composeContext
behavior/observation.js	観測形式の検証	createObservation
behavior/state-store.js	値と由来・鮮度の保持	apply、snapshot
behavior/world-model.js	規則で意味を導出	derive
behavior/drive-system.js	必要度と内部信号	observe、snapshot
behavior/spontaneous-behavior.js	自発候補の生成	evaluate
behavior/context-composer.js	判断文脈への投影	compose、projectWorld
behavior/environment-text.js	数値環境文の選別	hasEnvironmentReading
behavior/action-gate.js	操作許可と結果の管理	propose、close
behavior/event-store.js	観測ログの保存・復元	append、initialize
protocol/llm-response.js	応答値の厳密な監査	audit、validateActions
memory/memory-store.js	記憶の保存・採否・検索	addProposals、decide、retrieve
memory/normalize-existing.js	既存記憶の正規化	normalizeExistingMemories
social/social-state-store.js	関係・境界・約束	addProposals、context、isActionDenied

alice.jsは入出力の接続が集まるファイルです。状態の規則そのものと、HTTPや外部サービスの処理を区別して読んでください。記憶ストアと社会ストアも、実装された保存機能と、現在そこへ情報が届く経路を別々に確認します。

根拠（作業ツリー）：alice.js / src/behavior/index.js / src/protocol/llm-response.js / src/memory/normalize-existing.js / src/social/social-state-store.js

68 実装参照II：会話・Responses・記憶処理・画面

この表でもsrc/の接頭辞を省略しています。dashboard/とtools/はリポジトリ直下のディレクトリーです。

ファイル	責務	読み始める場所
live/session-service.js	セッション所有、イベント、同期、終了	start、handle、tick、stop
live/http-routes.js	LiveのHTTPとローカル制限	mountLiveRoutes
live/body-context.js	会話へ渡す身体の要約	bodyContext、会話スタイル
live/text-budget.js	文字列の予算	fitText、textChunks
live/kernel-bridge.js	委譲、カメラ、制御、記憶との接続	delegate、process、extract
live/backend-queue.js	一つのキュー内の直列化	submit、drain
live/responses-backend.js	API通信、応答解析、監査、検索	request、run、search
live/responses-schema.js	task別のJSON形式	responseFormats
live/conversation-store.js	原文ログと処理位置	append、groups、batch、commit
live/memory-wire.js	短い参照と根拠の復元	memoryConversation、restoreMemoryEvidence
live/memory-extractor.js	抽出、照合、採否、再試行	tick、extract
live/api-diagnostics.js	メタデータの追記と世代管理	record
dashboard/live-client.js	WebRTCと接続の所有	start、stop
dashboard/live-audio-monitor.js	音量・受信・再生の診断	sample、snapshot
dashboard/app.js	表示、手動操作、文字入力	update表示、活動、kokomi
tools/memory-list.js	記憶一覧の読出し	オプション解析と一覧表示

実際の公開名や補助関数はファイル内で確認してください。表は読解の入口を示す地図です。
docs/live-api-spec.mdも補助資料になりますが、動作の根拠は現在のソースと設定を優先しています。

根拠（作業ツリー）：src/live/session-service.js / src/live/kernel-bridge.js / src/live/memory-wire.js / dashboard/live-client.js / dashboard/live-audio-monitor.js / tools/memory-list.js

69 なぜこの機能が必要か：起きる問題から読む

部品名を覚えるより、避けたい問題と対策を組で理解すると、構造が定着します。

起こり得る問題	対応する仕組み	意味
機器ごとに入力形式が違う	観測の共通形式	同じ更新・保存経路へそろえる
古い測定を現在と思い込む	出所と鮮度	unknown / staleを扱える
生の数値だけでは判断しにくい	世界モデル	warmや在室という意味を作る
環境数値を毎回会話へ広げる	定性的な投影と選別	会話に必要な状態へ絞る
過去の依頼を新しい会話で再開する	日時付き履歴と新セッション境界	過去を参考情報と明示する
モデルが未対応操作を作る	スキーマと値の監査	形と実行可能性を検査する
同じ操作を連打する	ゲート、重複、クールダウン	実行の回数と間隔を制御する
訂正前の判断が遅れて届く	revisionと有効性検査	古いtaskを採用しない
成功したつもりで話す	intention / outcome	結果を別に確認して返す
検索本文が機体への指示になる	検索専用経路	出典の確認で処理を終える
記憶APIが会話の確認を止める	独立したキュー	制御と記憶の待ち時間を分ける
長い根拠IDをモデルが誤記する	u1等からカーネルが復元	原典の選択とIDの生成を分ける
原文があっても本文が飛躍する	別依頼での照合	候補本文の支持を確認する
保存途中の再実行で増殖する	preparedとsource_key	固定した仕事を重複せず再適用
失敗し続けてAPIを呼ぶ	待ち時間と連続失敗の保留	自動処理を制限する
音声が止まった原因が不明	ローカル音声統計とAPI診断	通信・再生・監査の段階を分ける

一つの対策ですべての問題は解決しません。たとえばゲートは操作の権限や頻度を扱い、照合は記憶本文の根拠を扱います。守る境界ごとに責任を分けているため、部品が複数あります。

根拠（作業ツリー）：src/behavior/index.js / src/behavior/action-gate.js / src/live/session-service.js / src/live/kernel-bridge.js / src/live/memory-extractor.js / src/live/api-diagnostics.js

70 出典・再現性：この説明が指す作業ツリー

この資料は、2026年11月8日のローカル作業ツリーを読み、コードの経路、同梱設定、共有制御プロンプト、既存テストを照合して作成しました。未コミット変更を含むため、コミット番号だけでは完全に同じ状態を再現できません。

識別項目	調査対象
ブランチ	codex/live-responses-kernel
HEAD	cbb47cf32ec3d778abe49c592092c738996cf4fe
参照範囲	エントリーポイント、src、dashboard、設定、test等
検証	Node.js 24.22.1 / 84テスト成功
個人データ	実際の会話・記憶の本文や.env秘密値は転載しない

付属ファイル

kernel_guide_ja.mdは本文の編集用原稿、build_kernel_guide.pyはPDFの生成処理です。
kernel_guide_sources.jsonは参照ソース54ファイルの相対パス・行数・SHA-256を保持します。
kernel_guide_validation.jsonはページ数、テストとPDF確認の記録です。

ソースの行番号はこの作業ツリーを基準にしています。後でファイルが変わると番号もずれるため、関数名とハッシュを併用してください。説明の例は架空で、実際のユーザーの会話ログではありません。

外部APIの概念を確認する公式資料

Structured Outputsは、JSONスキーマで応答の形を指定する仕組みの説明です。形が合うことは意味の正しさの保証ではなく、このカーネルは別に値と根拠を監査します。

Web searchは、検索ツールの呼出しとURL引用の説明です。この実装では完了した検索と引用を確認し、本文と出典を分けて返します。

Live delegationとVoice server controlsは、会話からの委譲とサーバー側の管理経路の説明です。今回の実装はクライアント委譲を受け、独立したResponses呼出しへつなぎます。

外部資料はAPIの概念理解を助けるものです。実装済みの経路・制限・設定値は、各ページのコード参照を根拠としています。

根拠（作業ツリー）：package.json / config.json / control_prompt.md / docs/live-api-spec.md / src/live/responses-backend.js / src/live/session-service.js